Okay, let's break down Python's built-in **Numeric Types** (`int` , `float` , `complex`) in detail, including their common operations and functions, with 10 examples for each, using the Edukron context where possible.

Python Built-in Numeric Data Types

Python offers three distinct built-in numeric types:

1. **`int` (Integer):** Represents whole numbers (positive, negative, or zero) without a fractional part. In Python 3, integers have arbitrary precision, meaning they can grow as large as your system's memory allows.
2. **`float` (Floating-Point Number):** Represents real numbers, including those with a fractional part indicated by a decimal point or scientific notation (e.g., `1.5e3` for 1500.0). Floats typically use the IEEE 754 standard for double-precision representation, which can sometimes lead to small precision inaccuracies.
3. **`complex` (Complex Number):** Represents numbers with a real and an imaginary part. The imaginary part is denoted by a `j` or `J` suffix (e.g., `3+5j`). While less common in general data metrics, they are crucial in many scientific and engineering domains.

Your examples:

- `total_students = 1500` is an `int` .
- `average_rating = 4.8` is a `float` .
- `growth_projection = complex(10, 3)` (or `10+3j`) is a `complex` number.

---

1. `int` (Integer)

Explanation: Integers represent whole numbers. They are fundamental for counting, indexing, and representing discrete quantities like the number of students, courses, or batches.

Common Operations & Functions:

Most operations on integers are performed using standard arithmetic operators or built-in functions applicable to various numeric types.

a) Addition (`+`)

Explanation: Adds two numbers. **Examples:**

```
# 1. Adding new students to the total
new_students_batch1 = 120
total_students = 1500
total_students = total_students + new_students_batch1
print(f"1. Total students after batch 1: {total_students}") # Output:
1620

# 2. Calculating total modules in two courses
ds_modules = 15
de_modules = 18
total_modules = ds_modules + de_modules
print(f"2. Total modules in DS & DE: {total_modules}") # Output: 33

# 3. Summing batch sizes
batch_size_1 = 50
batch_size_2 = 45
combined_batch_size = batch_size_1 + batch_size_2
print(f"3. Combined size of two batches: {combined_batch_size}") #
Output: 95

# 4. Adding zero doesn't change the value
initial_count = 100
final_count = initial_count + 0
print(f"4. Count after adding zero: {final_count}") # Output: 100

# 5. Adding a negative number (subtraction)
active_students = 1620
dropouts = -20 # Representing dropouts as negative addition
active_students = active_students + dropouts
print(f"5. Active students after dropouts: {active_students}") #
Output: 1600

# 6. Incrementing a counter
completed_projects = 5
completed_projects = completed_projects + 1
print(f"6. Completed projects incremented: {completed_projects}") #
Output: 6

# 7. Summing course durations (in weeks)
duration1 = 8
duration2 = 12
total_duration = duration1 + duration2
print(f"7. Total duration in weeks: {total_duration}") # Output: 20

# 8. Adding instructor counts
```

```python
ds_instructors = 3
de_instructors = 4
total_instructors = ds_instructors + de_instructors
print(f"8. Total instructors: {total_instructors}") # Output: 7

# 9. Calculating total assignments
assignments_part1 = 25
assignments_part2 = 30
total_assignments = assignments_part1 + assignments_part2
print(f"9. Total assignments: {total_assignments}") # Output: 55

# 10. Combining positive and negative changes
student_change = 50 + (-10) + 30
print(f"10. Net student change: {student_change}") # Output: 70
```

b) Subtraction (`-`)

Explanation: Subtracts the second number from the first. **Examples:**

```python
# 1. Calculating remaining seats
batch_capacity = 60
enrolled = 55
remaining_seats = batch_capacity - enrolled
print(f"1. Remaining seats: {remaining_seats}") # Output: 5

# 2. Calculating student dropouts
initial_students = 1500
final_students = 1485
dropouts = initial_students - final_students
print(f"2. Number of dropouts: {dropouts}") # Output: 15

# 3. Finding the difference in module counts
ds_modules = 15
de_modules = 18
module_difference = de_modules - ds_modules
print(f"3. Difference in modules: {module_difference}") # Output: 3

# 4. Subtracting zero
revenue = 50000
revenue_after_discount = revenue - 0
print(f"4. Revenue after zero discount: {revenue_after_discount}") #
Output: 50000

# 5. Resulting in a negative number
costs = 60000
profit = revenue - costs
print(f"5. Profit (can be negative): {profit}") # Output: -10000

# 6. Decrementing a counter
live_sessions_left = 10
```

```
live_sessions_left = live_sessions_left - 1
print(f"6. Live sessions left after one: {live_sessions_left}") #
Output: 9

# 7. Calculating years since launch
current_year = 2024
launch_year = 2020
years_active = current_year - launch_year
print(f"7. Edukron years active: {years_active}") # Output: 4

# 8. Finding difference between target and actual enrollment
target_enrollment = 2000
total_students = 1500
shortfall = target_enrollment - total_students
print(f"8. Enrollment shortfall: {shortfall}") # Output: 500

# 9. Subtracting a negative number (addition)
initial_score = 80
score_penalty = -10 # Penalty represented negatively
final_score = initial_score - score_penalty
print(f"9. Score after reversing penalty: {final_score}") # Output:
90

# 10. Days remaining in a course
total_days = 90
days_passed = 30
days_remaining = total_days - days_passed
print(f"10. Days remaining: {days_remaining}") # Output: 60
```

c) Multiplication (`*`)

Explanation: Multiplies two numbers. **Examples:**

```
# 1. Calculating total revenue
total_students = 1500
course_fee = 500
total_revenue = total_students * course_fee
print(f"1. Total potential revenue: ${total_revenue}") # Output:
750000

# 2. Calculating total hours for a course
modules = 15
hours_per_module = 4
total_hours = modules * hours_per_module
print(f"2. Total course hours: {total_hours}") # Output: 60

# 3. Scaling up projected growth
monthly_growth = 50 # students per month
months = 12
annual_growth_projection = monthly_growth * months
```

```python
print(f"3. Projected annual growth: {annual_growth_projection}") # Output: 600

# 4. Multiplying by zero
zero_enrollment_batches = 0
students_per_batch = 50
total_from_zero_batches = zero_enrollment_batches * students_per_batch
print(f"4. Students from zero batches: {total_from_zero_batches}") # Output: 0

# 5. Multiplying by one
active_courses = 3
multiplier = 1
result = active_courses * multiplier
print(f"5. Multiplying by one: {result}") # Output: 3

# 6. Calculating cost based on usage
api_calls = 10000
cost_per_call = 2 # cents
total_api_cost = api_calls * cost_per_call
print(f"6. Total API cost (cents): {total_api_cost}") # Output: 20000

# 7. Multiplying by a negative number
profit_per_student = 100
refunds = -5 # Representing 5 refunds
impact_on_profit = refunds * profit_per_student
print(f"7. Profit impact from refunds: ${impact_on_profit}") # Output: -500

# 8. Calculating total storage needed
students = 1500
storage_per_student_gb = 2
total_storage_gb = students * storage_per_student_gb
print(f"8. Total storage needed (GB): {total_storage_gb}") # Output: 3000

# 9. Area calculation (simple example)
width = 10
height = 5
area = width * height
print(f"9. Simple area: {area}") # Output: 50

# 10. Total quizzes across modules
modules = 15
quizzes_per_module = 3
total_quizzes = modules * quizzes_per_module
print(f"10. Total quizzes: {total_quizzes}") # Output: 45
```

d) Floor Division (`//`)

Explanation: Divides the first number by the second and rounds the result *down* to the nearest whole number (integer). **Examples:**

```
# 1. Calculating number of full batches
total_students = 157
students_per_batch = 50
full_batches = total_students // students_per_batch
print(f"1. Number of full batches: {full_batches}") # Output: 3

# 2. Distributing assignments equally
total_assignments = 47
num_instructors = 4
assignments_per_instructor = total_assignments // num_instructors
print(f"2. Assignments per instructor (min):
{assignments_per_instructor}") # Output: 11

# 3. Calculating completed weeks
days_elapsed = 30
days_per_week = 7
completed_weeks = days_elapsed // days_per_week
print(f"3. Completed weeks: {completed_weeks}") # Output: 4

# 4. Dividing by a larger number
small_group = 10
large_capacity = 100
result = small_group // large_capacity
print(f"4. Result of 10 // 100: {result}") # Output: 0

# 5. Floor division with zero numerator
zero_students = 0
batch_size = 50
result = zero_students // batch_size
print(f"5. Result of 0 // 50: {result}") # Output: 0

# 6. Floor division with negative numbers (rounds down)
debt = -100
payments_per_month = 30
months_of_full_payments = debt // payments_per_month # How many full
payment periods fit into the debt? Needs care with sign.
print(f"6. Result of -100 // 30: {months_of_full_payments}") #
Output: -4

# 7. Another negative example
val1 = -10
val2 = 3
result = val1 // val2
```

```
print(f"7. Result of -10 // 3: {result}") # Output: -4

# 8. Positive numerator, negative denominator
val1 = 10
val2 = -3
result = val1 // val2
print(f"8. Result of 10 // -3: {result}") # Output: -4

# 9. Calculating full sets of resources
total_licenses = 125
licenses_per_set = 10
full_sets = total_licenses // licenses_per_set
print(f"9. Full sets of licenses: {full_sets}") # Output: 12

# 10. Integer part of an average (if calculated stepwise)
total_score = 485
num_students = 50
integer_part_avg = total_score // num_students
print(f"10. Integer part of average score: {integer_part_avg}") #
Output: 9
```

e) Modulo (`%`)

Explanation: Returns the *remainder* of the division of the first number by the second. **Examples:**

```
# 1. Finding remaining students after forming full batches
total_students = 157
students_per_batch = 50
remaining_students = total_students % students_per_batch
print(f"1. Students for the last (incomplete) batch:
{remaining_students}") # Output: 7

# 2. Finding leftover assignments
total_assignments = 47
num_instructors = 4
leftover_assignments = total_assignments % num_instructors
print(f"2. Leftover assignments to distribute:
{leftover_assignments}") # Output: 3

# 3. Finding remaining days in the current week
days_elapsed = 30
days_per_week = 7
day_in_week = days_elapsed % days_per_week # (Assuming day 0 is the
start)
print(f"3. Day number within the current week (0-6): {day_in_week}")
# Output: 2

# 4. Checking for even or odd numbers
```

```python
student_id = 1501
is_odd = student_id % 2
print(f"4. Is student ID {student_id} odd (1 means yes)? {is_odd}") # Output: 1
batch_id = 102
is_even = batch_id % 2
print(f"   Is batch ID {batch_id} even (0 means yes)? {is_even}") # Output: 0

# 5. Remainder when dividing by a larger number
small_group = 10
large_capacity = 100
remainder = small_group % large_capacity
print(f"5. Remainder of 10 % 100: {remainder}") # Output: 10

# 6. Modulo with zero numerator
zero_students = 0
batch_size = 50
remainder = zero_students % batch_size
print(f"6. Remainder of 0 % 50: {remainder}") # Output: 0

# 7. Modulo with negative numerator (result takes sign of divisor)
debt_remaining = -7
monthly_payment = 30
result = debt_remaining % monthly_payment
print(f"7. Result of -7 % 30: {result}") # Output: 23 (because -7 = -1*30 + 23)

# 8. Another negative example
val1 = -10
val2 = 3
remainder = val1 % val2
print(f"8. Result of -10 % 3: {remainder}") # Output: 2 (because -10 = -4*3 + 2)

# 9. Positive numerator, negative denominator
val1 = 10
val2 = -3
remainder = val1 % val2
print(f"9. Result of 10 % -3: {remainder}") # Output: -2 (because 10 = -4*-3 + (-2))

# 10. Finding leftover licenses
total_licenses = 125
licenses_per_set = 10
leftover_licenses = total_licenses % licenses_per_set
print(f"10. Leftover licenses: {leftover_licenses}") # Output: 5
```

f) Exponentiation (`**`)

Explanation: Raises the first number to the power of the second number.

Examples:

```
# 1. Modeling simplified exponential student growth (base *
factor^periods)
initial_students = 100
growth_factor = 2
periods = 3
projected_students = initial_students * (growth_factor ** periods)
print(f"1. Projected students after {periods} periods:
{projected_students}") # Output: 800

# 2. Calculating squares (power of 2)
side_length = 5
area = side_length ** 2
print(f"2. Area of square: {area}") # Output: 25

# 3. Calculating cubes (power of 3)
edge_length = 3
volume = edge_length ** 3
print(f"3. Volume of cube: {volume}") # Output: 27

# 4. Power of zero (always 1, except 0**0 is debated but 1 in Python)
base = 10
result = base ** 0
print(f"4. 10 to the power of 0: {result}") # Output: 1
zero_base = 0
result_zero = zero_base ** 0
print(f"   0 to the power of 0: {result_zero}") # Output: 1

# 5. Power of one (always the base itself)
base = total_students # 1500
result = base ** 1
print(f"5. {base} to the power of 1: {result}") # Output: 1500

# 6. Large powers (possible due to arbitrary precision)
large_power = 2 ** 100
print(f"6. 2 to the power of 100 (first few digits):
{str(large_power)[:10]}...") # Output: 1267650600...

# 7. Negative base, even power (positive result)
base = -5
power = 2
result = base ** power
print(f"7. (-5) to the power of 2: {result}") # Output: 25
```

```
# 8. Negative base, odd power (negative result)
base = -5
power = 3
result = base ** power
print(f"8. (-5) to the power of 3: {result}") # Output: -125

# 9. Using exponentiation for bit shifting (2**n is like 1 << n)
bit_shift_equivalent = 2 ** 3
print(f"9. 2 to the power of 3: {bit_shift_equivalent}") # Output: 8
(same as 1 << 3)

# 10. Calculating compound interest (simplified, int only)
principal = 10000
rate_percent = 5 # Integer percent
periods = 3
# Formula P * (1 + r)^n -> Needs float usually, but simplified:
# This is NOT accurate finance, just showing the operator
approx_value = principal * (100 + rate_percent) ** periods // (100
** periods)
print(f"10. Simplified compound interest approx: {approx_value}") #
Output: 11576 (approximate)
```

g) `abs()`

Explanation: Returns the absolute (non-negative) value of a number. **Examples:**

```
# 1. Getting absolute change in students
student_change = -50 # 50 students left
abs_change = abs(student_change)
print(f"1. Absolute student change: {abs_change}") # Output: 50

# 2. Absolute value of a positive number
positive_count = 1500
abs_count = abs(positive_count)
print(f"2. Absolute value of 1500: {abs_count}") # Output: 1500

# 3. Absolute value of zero
zero_value = 0
abs_zero = abs(zero_value)
print(f"3. Absolute value of 0: {abs_zero}") # Output: 0

# 4. Ensuring a difference is positive
score1 = 85
score2 = 92
score_diff = abs(score1 - score2)
print(f"4. Absolute difference between scores: {score_diff}") #
Output: 7

# 5. Absolute error calculation
estimated_students = 1450
```

```
actual_students = 1500
abs_error = abs(estimated_students - actual_students)
print(f"5. Absolute error in estimation: {abs_error}") # Output: 50

# 6. Using abs() in calculations
value = -10
result = abs(value) * 5
print(f"6. Calculation with abs(): {result}") # Output: 50

# 7. Absolute value from a calculation result
net_profit = 10000 - 15000 # -5000
abs_profit_or_loss = abs(net_profit)
print(f"7. Magnitude of profit/loss: {abs_profit_or_loss}") # Output:
5000

# 8. Applying to batch id difference
batch_id1 = 101
batch_id2 = 105
id_distance = abs(batch_id1 - batch_id2)
print(f"8. Distance between batch IDs: {id_distance}") # Output: 4

# 9. Absolute value of a large negative number
large_neg = -999999999
abs_large = abs(large_neg)
print(f"9. Absolute value of large negative: {abs_large}") # Output:
999999999

# 10. Absolute value within a conditional check (conceptual)
deviation = -5
if abs(deviation) > 3:
    print(f"10. Deviation {deviation} magnitude is significant.") #
Output: Deviation -5 magnitude is significant.
```

h) `pow(base, exp[, mod])`

Explanation: A built-in function alternative to `**`. `pow(x, y)` is equivalent to `x ** y`. It also supports an optional third argument for efficient modular exponentiation `(x ** y) % z`. **Examples:**

```
# 1. Simple power calculation
result = pow(10, 3)
print(f"1. pow(10, 3): {result}") # Output: 1000

# 2. Equivalent to growth example above
initial_students = 100
growth_factor = 2
periods = 3
projected_students = initial_students * pow(growth_factor, periods)
print(f"2. Projected students using pow(): {projected_students}") #
```

```
Output: 800

# 3. Power of 2
side_length = 5
area = pow(side_length, 2)
print(f"3. Area using pow(): {area}") # Output: 25

# 4. Power of 0
base = total_students # 1500
result = pow(base, 0)
print(f"4. pow({base}, 0): {result}") # Output: 1

# 5. Power of 1
base = 42
result = pow(base, 1)
print(f"5. pow(42, 1): {result}") # Output: 42

# 6. Negative base, even power
result = pow(-5, 2)
print(f"6. pow(-5, 2): {result}") # Output: 25

# 7. Negative base, odd power
result = pow(-5, 3)
print(f"7. pow(-5, 3): {result}") # Output: -125

# 8. Modular exponentiation: (3 ** 4) % 5
result_mod = pow(3, 4, 5) # 3^4 = 81. 81 % 5 = 1
print(f"8. pow(3, 4, 5): {result_mod}") # Output: 1

# 9. Modular exponentiation: (2 ** 10) % 7
result_mod_large = pow(2, 10, 7) # 2^10 = 1024. 1024 % 7 = 2
print(f"9. pow(2, 10, 7): {result_mod_large}") # Output: 2

# 10. Modular exponentiation with negative base (handle carefully
based on definition)
# pow(-2, 3, 5) -> (-2)^3 = -8. (-8) % 5 -> Python gives 2 (-8 = -2*5
+ 2)
result_mod_neg = pow(-2, 3, 5)
print(f"10. pow(-2, 3, 5): {result_mod_neg}") # Output: 2
```

i) `divmod(a, b)`

Explanation: Takes two numbers (a, b) and returns a pair of numbers (a tuple) consisting of their quotient (`a // b`) and remainder (`a % b`). **Examples:**

```
# 1. Getting full batches and remaining students simultaneously
total_students = 157
students_per_batch = 50
batches, remaining = divmod(total_students, students_per_batch)
```

```
print(f"1. Batches: {batches}, Remaining Students: {remaining}") #
Output: Batches: 3, Remaining Students: 7

# 2. Distributing assignments and finding leftovers
total_assignments = 47
num_instructors = 4
assign_per, leftover_assign = divmod(total_assignments,
num_instructors)
print(f"2. Assignments per instructor: {assign_per}, Leftovers:
{leftover_assign}") # Output: Assignments per instructor: 11,
Leftovers: 3

# 3. Calculating weeks and remaining days
days_elapsed = 30
days_per_week = 7
weeks, rem_days = divmod(days_elapsed, days_per_week)
print(f"3. Weeks: {weeks}, Remaining Days: {rem_days}") # Output:
Weeks: 4, Remaining Days: 2

# 4. Dividing by a larger number
small_group = 10
large_capacity = 100
quotient, remainder = divmod(small_group, large_capacity)
print(f"4. divmod(10, 100): Quotient={quotient},
Remainder={remainder}") # Output: Quotient=0, Remainder=10

# 5. Dividing zero
zero_students = 0
batch_size = 50
quotient, remainder = divmod(zero_students, batch_size)
print(f"5. divmod(0, 50): Quotient={quotient},
Remainder={remainder}") # Output: Quotient=0, Remainder=0

# 6. Negative numerator
debt = -100
payments_per_month = 30
q, r = divmod(debt, payments_per_month)
print(f"6. divmod(-100, 30): Quotient={q}, Remainder={r}") # Output:
Quotient=-4, Remainder=20

# 7. Another negative numerator example
q, r = divmod(-10, 3)
print(f"7. divmod(-10, 3): Quotient={q}, Remainder={r}") # Output:
Quotient=-4, Remainder=2

# 8. Positive numerator, negative denominator
q, r = divmod(10, -3)
print(f"8. divmod(10, -3): Quotient={q}, Remainder={r}") # Output:
Quotient=-4, Remainder=-2
```

```
# 9. Calculating full sets and leftovers
total_licenses = 125
licenses_per_set = 10
sets, leftover_licenses = divmod(total_licenses, licenses_per_set)
print(f"9. Full sets: {sets}, Leftover licenses:
{leftover_licenses}") # Output: Full sets: 12, Leftover licenses: 5

# 10. Converting total minutes to hours and minutes
total_minutes = 135
minutes_per_hour = 60
hours, minutes = divmod(total_minutes, minutes_per_hour)
print(f"10. {total_minutes} mins = {hours} hours and {minutes}
minutes") # Output: 135 mins = 2 hours and 15 minutes
```

j) `int()`

Explanation: While also the type constructor, `int()` can be used as a function to convert values from other types (like `float` or compatible strings) to an integer. It truncates towards zero when converting floats. **Examples:**

```
# 1. Converting average rating (float) to int (truncates)
average_rating = 4.8
rating_int = int(average_rating)
print(f"1. Average rating as int: {rating_int}") # Output: 4

# 2. Converting a string representation of an integer
total_students_str = "1500"
total_students_int = int(total_students_str)
print(f"2. Total students from string: {total_students_int}") #
Output: 1500

# 3. Converting a negative float string
neg_float_str = "-10.5"
neg_int = int(float(neg_float_str)) # Need float() first if string
has decimal
print(f"3. Integer from '-10.5': {neg_int}") # Output: -10

# 4. Converting boolean True to int
is_active_int = int(True)
print(f"4. Integer value of True: {is_active_int}") # Output: 1

# 5. Converting boolean False to int
is_full_int = int(False)
print(f"5. Integer value of False: {is_full_int}") # Output: 0

# 6. Converting a positive float
positive_float = 99.99
positive_int = int(positive_float)
print(f"6. Integer from 99.99: {positive_int}") # Output: 99
```

```
# 7. Converting a negative float
negative_float = -0.1
negative_int = int(negative_float)
print(f"7. Integer from -0.1: {negative_int}") # Output: 0

# 8. Converting string with base (e.g., binary)
binary_str = "1101" # Represents 13
decimal_val = int(binary_str, 2)
print(f"8. Integer from binary '1101': {decimal_val}") # Output: 13

# 9. Converting string with base (e.g., hexadecimal)
hex_str = "FF" # Represents 255
decimal_val_hex = int(hex_str, 16)
print(f"9. Integer from hex 'FF': {decimal_val_hex}") # Output: 255

# 10. Attempting invalid conversion (raises ValueError)
try:
    invalid_int = int("Edukron")
except ValueError as e:
    print(f"10. Error converting 'Edukron' to int: {e}") # Output:
Error converting 'Edukron' to int: invalid literal for int() with
base 10: 'Edukron'
```

---

2. `float` (Floating-Point Number)

Explanation: Floats represent numbers with a decimal point. They are essential for measurements, averages, percentages, financial calculations, and any quantity that might not be a whole number, like `average_rating`. Be mindful of potential small precision errors inherent in binary floating-point representation.

Common Operations & Functions/Methods:

Floats support the same arithmetic operators (`+`, `-`, `*`, `/`, `//`, `%`, `**`) as integers, but the results are typically floats (especially with `/`). They also work with `abs()`, `pow()`, `divmod()`, and have specific methods.

a) Arithmetic Operations (`+`, `-`, `*`, `/`)

Explanation: Standard arithmetic. Division (`/`) always results in a float, even if the mathematical result is a whole number. **Examples (showing `/` specifically):**

```
# 1. Calculating average score precisely
total_score = 485.0 # Use float for precision
num_students = 50
```

```python
average_score = total_score / num_students
print(f"1. Precise average score: {average_score}") # Output: 9.7

# 2. Calculating completion percentage
completed_modules = 7.0
total_modules = 15
completion_rate = (completed_modules / total_modules) * 100
print(f"2. Course completion rate: {completion_rate:.2f}%") # Output:
46.67% (formatted)

# 3. Dividing integers results in float
revenue = 750000
total_students = 1500
revenue_per_student = revenue / total_students
print(f"3. Revenue per student: {revenue_per_student}") # Output:
500.0 (Note the .0)

# 4. Simple division
val1 = 10.0
val2 = 4.0
result = val1 / val2
print(f"4. 10.0 / 4.0 = {result}") # Output: 2.5

# 5. Division involving zero
result = 0.0 / 5.0
print(f"5. 0.0 / 5.0 = {result}") # Output: 0.0

# 6. Division by small number
value = 1.0
divisor = 0.001
result = value / divisor
print(f"6. 1.0 / 0.001 = {result}") # Output: 1000.0

# 7. Using division in scaling
original_rating = 4.8
scale_factor = 2.0 # Example: Adjusting scale
scaled_rating = original_rating / scale_factor
print(f"7. Scaled rating: {scaled_rating}") # Output: 2.4

# 8. Negative division
val1 = -15.0
val2 = 2.0
result = val1 / val2
print(f"8. -15.0 / 2.0 = {result}") # Output: -7.5

# 9. Division resulting in repeating decimal (shows precision limits)
result = 1.0 / 3.0
print(f"9. 1.0 / 3.0 = {result}") # Output: 0.3333333333333333

# 10. Calculating average monthly growth
```

```
total_growth = 55.5 # e.g. average increase over period
months = 6
avg_monthly = total_growth / months
print(f"10. Average monthly growth: {avg_monthly}") # Output: 9.25
```

(Examples for `+`, `-`, `are similar to` int` but operate on/produce floats)

b) Floor Division (`//`) and Modulo (`%`) on Floats

Explanation: These operators also work with floats, following the same logic (floor division rounds down, modulo gives remainder), but the results can be floats. `divmod()` also works. **Examples (using `//`):**

```
# 1. How many full 'rating points' in average_rating
average_rating = 4.8
full_points = average_rating // 1.0
print(f"1. Full rating points in {average_rating}: {full_points}") #
Output: 4.0

# 2. Number of full $10 units in a price
price = 49.95
units_of_10 = price // 10.0
print(f"2. Full $10 units in {price}: {units_of_10}") # Output: 4.0

# 3. Dividing floats with floor division
val1 = 10.5
val2 = 2.1
result = val1 // val2
print(f"3. 10.5 // 2.1 = {result}") # Output: 5.0

# 4. Negative float floor division
val1 = -10.5
val2 = 2.1
result = val1 // val2
print(f"4. -10.5 // 2.1 = {result}") # Output: -6.0 (rounds down
towards negative infinity)

# 5. Using very small divisor
val1 = 1.0
val2 = 0.3
result = val1 // val2
print(f"5. 1.0 // 0.3 = {result}") # Output: 3.0

# 6. Floor division by 1.0
val = 7.8
result = val // 1.0
print(f"6. 7.8 // 1.0 = {result}") # Output: 7.0

# 7. Floor division result is zero
val1 = 2.5
```

```
val2 = 3.0
result = val1 // val2
print(f"7. 2.5 // 3.0 = {result}") # Output: 0.0

# 8. Calculating full weeks from fractional days
days = 30.5
days_per_week = 7.0
full_weeks = days // days_per_week
print(f"8. Full weeks in {days} days: {full_weeks}") # Output: 4.0

# 9. Large numbers
val1 = 1000.0
val2 = 150.5
result = val1 // val2
print(f"9. 1000.0 // 150.5 = {result}") # Output: 6.0

# 10. Floor division involving integers (promotes to float)
result = 10 // 3.0
print(f"10. 10 // 3.0 = {result}") # Output: 3.0
```

(`%` and `divmod` examples follow similarly)

c) `round(number[, ndigits])`

Explanation: Rounds a number to a specified number of decimal places (`ndigits`). If `ndigits` is omitted or `None`, it rounds to the nearest integer (returns an `int`). Uses "round half to even" strategy for values exactly halfway between two numbers (e.g., `round(2.5)` is 2, `round(3.5)` is 4). **Examples:**

```
# 1. Rounding the average rating to one decimal place
average_rating = 4.8356
rounded_rating_1dp = round(average_rating, 1)
print(f"1. Average rating rounded to 1dp: {rounded_rating_1dp}") #
Output: 4.8

# 2. Rounding the average rating to the nearest integer
rounded_rating_int = round(average_rating)
print(f"2. Average rating rounded to nearest int:
{rounded_rating_int}") # Output: 5

# 3. Rounding completion rate to two decimal places
completion_rate = 46.6666666
rounded_rate_2dp = round(completion_rate, 2)
print(f"3. Completion rate rounded to 2dp: {rounded_rate_2dp}") #
Output: 46.67

# 4. Rounding to zero decimal places (returns float)
price = 49.95
rounded_price_0dp = round(price, 0)
print(f"4. Price rounded to 0dp: {rounded_price_0dp}") # Output: 50.0
```

```python
# 5. Round half to even: 2.5
result = round(2.5)
print(f"5. round(2.5): {result}") # Output: 2

# 6. Round half to even: 3.5
result = round(3.5)
print(f"6. round(3.5): {result}") # Output: 4

# 7. Rounding a negative number
neg_value = -4.8356
rounded_neg_1dp = round(neg_value, 1)
print(f"7. -4.8356 rounded to 1dp: {rounded_neg_1dp}") # Output: -4.8

# 8. Rounding to nearest integer (negative)
rounded_neg_int = round(neg_value)
print(f"8. -4.8356 rounded to nearest int: {rounded_neg_int}") #
Output: -5

# 9. Rounding to negative digits (rounds to tens, hundreds...)
total_students_precise = 1537.5
rounded_to_tens = round(total_students_precise, -1)
print(f"9. Students rounded to nearest 10: {rounded_to_tens}") #
Output: 1540

# 10. Rounding to nearest hundred
revenue = 753450.75
rounded_revenue_hundreds = round(revenue, -2)
print(f"10. Revenue rounded to nearest 100:
{rounded_revenue_hundreds}") # Output: 753500
```

d) `float.is_integer()`

Explanation: Method called on a float object. Returns `True` if the float value represents a whole number (has no fractional part), `False` otherwise. **Examples:**

```python
# 1. Check if average rating is an integer
average_rating = 4.8
print(f"1. Is {average_rating} an integer?
{average_rating.is_integer()}") # Output: False

# 2. Check a float that is an integer
revenue_per_student = 500.0
print(f"2. Is {revenue_per_student} an integer?
{revenue_per_student.is_integer()}") # Output: True

# 3. Check zero
zero_float = 0.0
print(f"3. Is {zero_float} an integer? {zero_float.is_integer()}") #
```

```
Output: True

# 4. Check a negative integer float
neg_int_float = -10.0
print(f"4. Is {neg_int_float} an integer?
{neg_int_float.is_integer()}") # Output: True

# 5. Check a negative non-integer float
neg_non_int_float = -10.5
print(f"5. Is {neg_non_int_float} an integer?
{neg_non_int_float.is_integer()}") # Output: False

# 6. Result of a calculation
result = 10.0 / 2.0 # 5.0
print(f"6. Is result of 10.0/2.0 an integer? {result.is_integer()}")
# Output: True

# 7. Result of another calculation
result = 10.0 / 3.0 # 3.333...
print(f"7. Is result of 10.0/3.0 an integer? {result.is_integer()}")
# Output: False

# 8. Float representation of total students
float_students = float(1500)
print(f"8. Is float(1500) an integer?
{float_students.is_integer()}") # Output: True

# 9. Very small fractional part (may be False due to precision)
small_frac = 1.0000000000000001
print(f"9. Is {small_frac} an integer? {small_frac.is_integer()}") #
Output: False

# 10. A float resulting from rounding
rounded_val = round(4.8) # Becomes int 5, but if result was float
like round(4.8, 0) -> 5.0
rounded_float = round(4.95, 0) # 5.0
print(f"10. Is round(4.95, 0) an integer?
{rounded_float.is_integer()}") # Output: True
```

e) `float.as_integer_ratio()`

Explanation: Method called on a float object. Returns a pair (tuple) of two integers whose ratio is exactly equal to the float value. Useful for representing floats precisely as fractions. The fraction is always in simplest form (lowest terms).

Examples:

```
# 1. Ratio for 0.5
ratio = (0.5).as_integer_ratio()
print(f"1. Ratio for 0.5: {ratio}") # Output: (1, 2)
```

```python
# 2. Ratio for average rating (shows potential complexity)
average_rating = 4.8
ratio = average_rating.as_integer_ratio()
# 4.8 = 48/10 = 24/5
print(f"2. Ratio for {average_rating}: {ratio}") # May show large
numbers due to precision: (5404319552844595, 1125899906842624) ->
this IS 4.8
# Let's try with a simpler representation if possible
ratio_simple = (4.75).as_integer_ratio() # 4.75 = 19/4
print(f"   Ratio for 4.75: {ratio_simple}") # Output: (19, 4)

# 3. Ratio for an integer float
ratio = (500.0).as_integer_ratio()
print(f"3. Ratio for 500.0: {ratio}") # Output: (500, 1)

# 4. Ratio for 0.25
ratio = (0.25).as_integer_ratio()
print(f"4. Ratio for 0.25: {ratio}") # Output: (1, 4)

# 5. Ratio for a negative float
ratio = (-0.75).as_integer_ratio()
print(f"5. Ratio for -0.75: {ratio}") # Output: (-3, 4)

# 6. Ratio for zero
ratio = (0.0).as_integer_ratio()
print(f"6. Ratio for 0.0: {ratio}") # Output: (0, 1)

# 7. Ratio demonstrating precision limits (0.1 is not exact in
binary)
ratio = (0.1).as_integer_ratio()
print(f"7. Ratio for 0.1 (precision): {ratio}") # Output:
(3602879701896397, 36028797018963968) - NOT (1, 10)

# 8. Ratio for 1.5
ratio = (1.5).as_integer_ratio()
print(f"8. Ratio for 1.5: {ratio}") # Output: (3, 2)

# 9. Ratio for a large float
large_float = 12345.6789
ratio = large_float.as_integer_ratio()
print(f"9. Ratio for {large_float} (can be large): {ratio}") #
Output: (some large integers)

# 10. Ratio from a calculation result
result = 10.0 / 8.0 # 1.25
ratio = result.as_integer_ratio()
print(f"10. Ratio for 1.25: {ratio}") # Output: (5, 4)
```

f) `float()`

Explanation: The type constructor, can also be used as a function to convert other types (like `int` or compatible strings) to a float. **Examples:**

```
# 1. Converting total students (int) to float
total_students = 1500
students_float = float(total_students)
print(f"1. Total students as float: {students_float}") # Output:
1500.0

# 2. Converting integer 0 to float
zero_float = float(0)
print(f"2. Integer 0 as float: {zero_float}") # Output: 0.0

# 3. Converting a string representation of a float
average_rating_str = "4.8"
average_rating_float = float(average_rating_str)
print(f"3. Average rating from string: {average_rating_float}") #
Output: 4.8

# 4. Converting a string representation of an integer
int_str = "100"
int_str_float = float(int_str)
print(f"4. Integer string '100' as float: {int_str_float}") # Output:
100.0

# 5. Converting a string with scientific notation
sci_str = "1.5e3" # 1.5 * 10^3
sci_float = float(sci_str)
print(f"5. Float from '1.5e3': {sci_float}") # Output: 1500.0

# 6. Converting boolean True to float
true_float = float(True)
print(f"6. Float value of True: {true_float}") # Output: 1.0

# 7. Converting boolean False to float
false_float = float(False)
print(f"7. Float value of False: {false_float}") # Output: 0.0

# 8. Converting special strings 'inf', '-inf', 'nan'
inf_float = float('inf')
print(f"8. Float from 'inf': {inf_float}") # Output: inf
neg_inf_float = float('-inf')
print(f"   Float from '-inf': {neg_inf_float}") # Output: -inf
nan_float = float('nan')
print(f"   Float from 'nan': {nan_float}") # Output: nan
```

```
# 9. Converting a negative integer string
neg_int_str = "-50"
neg_int_float = float(neg_int_str)
print(f"9. Float from '-50': {neg_int_float}") # Output: -50.0

# 10. Attempting invalid conversion (raises ValueError)
try:
    invalid_float = float("Edukron 4.8")
except ValueError as e:
    print(f"10. Error converting 'Edukron 4.8' to float: {e}") #
Output: Error ... could not convert string to float: 'Edukron 4.8'
```

---

3. `complex` (Complex Number)

Explanation: Complex numbers have a real part and an imaginary part (suffixed with `j`). They are created using `complex(real, imag)` or by writing the literal form like `10+3j`. While your `growth_projection = complex(10, 3)` example is valid, applying complex numbers meaningfully to typical Edukron metrics like student count or ratings is abstract; they are more common in fields like electrical engineering, signal processing, quantum mechanics, and certain mathematical visualizations. We'll use the `growth_projection` variable and other examples.

Common Operations & Attributes/Methods:

Complex numbers support standard arithmetic operations (`+`, `-`, `*`, `/`, `**`). They also work with `abs()` (which calculates the magnitude), `pow()`, and have specific attributes `.real`, `.imag`, and a method `.conjugate()`.

a) Arithmetic Operations (`+`, `-`, `*`, `/`)

Explanation: Standard arithmetic, following the rules of complex number math.
Examples (using `*`):

```
# Let's define two complex projections/factors
growth_projection = complex(10, 3) # 10 + 3j
adjustment_factor = complex(1, -0.5) # 1 - 0.5j

# 1. Multiplying two complex numbers
# (a+bj)(c+dj) = (ac-bd) + (ad+bc)j
# (10+3j)(1-0.5j) = (10*1 - 3*(-0.5)) + (10*(-0.5) + 3*1)j
#                 = (10 + 1.5) + (-5 + 3)j = 11.5 - 2j
result = growth_projection * adjustment_factor
print(f"1. {growth_projection} * {adjustment_factor} = {result}") #
Output: (11.5-2j)
```

```python
# 2. Multiplying by a real number (int or float)
scale_factor = 2
scaled_projection = growth_projection * scale_factor
print(f"2. {growth_projection} * {scale_factor} =
{scaled_projection}") # Output: (20+6j)

# 3. Multiplying by pure imaginary number (j)
imag_factor = 1j
rotated_projection = growth_projection * imag_factor # Rotates by 90
deg
# (10+3j)*j = 10j + 3j^2 = 10j - 3 = -3 + 10j
print(f"3. {growth_projection} * {imag_factor} =
{rotated_projection}") # Output: (-3+10j)

# 4. Squaring a complex number
squared_projection = growth_projection ** 2
# (10+3j)^2 = 100 + 2*10*3j + (3j)^2 = 100 + 60j - 9 = 91 + 60j
print(f"4. ({growth_projection})^2 = {squared_projection}") # Output:
(91+60j)

# 5. Multiplying by zero complex number
zero_complex = complex(0, 0)
result = growth_projection * zero_complex
print(f"5. {growth_projection} * {zero_complex} = {result}") #
Output: 0j

# 6. Multiplying conjugates: (a+bj)(a-bj) = a^2 + b^2 (a real number)
conjugate = growth_projection.conjugate() # 10-3j
result = growth_projection * conjugate
# (10+3j)(10-3j) = 10^2 + 3^2 = 100 + 9 = 109
print(f"6. {growth_projection} * {conjugate} = {result}") # Output:
(109+0j)

# 7. Multiplying complex numbers involving negatives
c1 = complex(-2, 1) # -2 + j
c2 = complex(1, -3) # 1 - 3j
# (-2+j)(1-3j) = (-2*1 - 1*(-3)) + (-2*(-3) + 1*1)j
#              = (-2 + 3) + (6 + 1)j = 1 + 7j
result = c1 * c2
print(f"7. {c1} * {c2} = {result}") # Output: (1+7j)

# 8. Multiplication resulting in pure real
c1 = 1+1j
c2 = 1-1j
result = c1 * c2 # (1+1j)(1-1j) = 1^2 + 1^2 = 2
print(f"8. {c1} * {c2} = {result}") # Output: (2+0j)

# 9. Multiplication resulting in pure imaginary
c1 = 1+1j
c2 = 1+1j
```

```
result = c1 * c2 # (1+1j)^2 = 1 + 2j + j^2 = 1 + 2j - 1 = 2j
print(f"9. {c1} * {c2} = {result}") # Output: 2j

# 10. Multiplying by 1+0j (identity)
identity = complex(1, 0)
result = growth_projection * identity
print(f"10. {growth_projection} * {identity} = {result}") # Output:
(10+3j)
```

(Examples for `+`, `-`, `/` follow complex arithmetic rules)

b) `.real`

Explanation: An attribute that returns the real part of the complex number as a float. **Examples:**

```
growth_projection = 10+3j
# 1. Get real part of growth_projection
real_part = growth_projection.real
print(f"1. Real part of {growth_projection}: {real_part}") # Output:
10.0

# 2. Real part of a pure imaginary number
imag_num = 5j
real_part = imag_num.real
print(f"2. Real part of {imag_num}: {real_part}") # Output: 0.0

# 3. Real part of a real number (represented as complex)
real_num = complex(total_students, 0) # complex(1500, 0)
real_part = real_num.real
print(f"3. Real part of {real_num}: {real_part}") # Output: 1500.0

# 4. Real part of a negative real component
neg_real_complex = -5+2j
real_part = neg_real_complex.real
print(f"4. Real part of {neg_real_complex}: {real_part}") # Output:
-5.0

# 5. Real part of zero complex number
zero_complex = 0j
real_part = zero_complex.real
print(f"5. Real part of {zero_complex}: {real_part}") # Output: 0.0

# 6. Using .real in a calculation
result = growth_projection.real * 2
print(f"6. Real part * 2: {result}") # Output: 20.0

# 7. Real part of a complex number with float components
float_complex = complex(average_rating, -1.5) # complex(4.8, -1.5)
real_part = float_complex.real
```

```
print(f"7. Real part of {float_complex}: {real_part}") # Output: 4.8

# 8. Real part from a calculation result
c1 = 1+2j
c2 = 3+4j
sum_c = c1 + c2 # 4+6j
real_part = sum_c.real
print(f"8. Real part of sum ({sum_c}): {real_part}") # Output: 4.0

# 9. Real part of conjugate
conjugate = growth_projection.conjugate() # 10-3j
real_part = conjugate.real
print(f"9. Real part of conjugate ({conjugate}): {real_part}") #
Output: 10.0 (same as original)

# 10. Real part when imag is zero
c = 7+0j
real_part = c.real
print(f"10. Real part of {c}: {real_part}") # Output: 7.0
```

c) `.imag`

Explanation: An attribute that returns the imaginary part of the complex number as a float. **Examples:**

```
growth_projection = 10+3j
# 1. Get imaginary part of growth_projection
imag_part = growth_projection.imag
print(f"1. Imaginary part of {growth_projection}: {imag_part}") #
Output: 3.0

# 2. Imaginary part of a pure imaginary number
imag_num = 5j
imag_part = imag_num.imag
print(f"2. Imaginary part of {imag_num}: {imag_part}") # Output: 5.0

# 3. Imaginary part of a real number (represented as complex)
real_num = complex(total_students, 0) # complex(1500, 0)
imag_part = real_num.imag
print(f"3. Imaginary part of {real_num}: {imag_part}") # Output: 0.0

# 4. Imaginary part of a complex number with negative imaginary
component
neg_imag_complex = 4-7j
imag_part = neg_imag_complex.imag
print(f"4. Imaginary part of {neg_imag_complex}: {imag_part}") #
Output: -7.0

# 5. Imaginary part of zero complex number
```

```
zero_complex = 0j
imag_part = zero_complex.imag
print(f"5. Imaginary part of {zero_complex}: {imag_part}") # Output:
0.0

# 6. Using .imag in a calculation
result = growth_projection.imag * 10
print(f"6. Imaginary part * 10: {result}") # Output: 30.0

# 7. Imaginary part of a complex number with float components
float_complex = complex(average_rating, -1.5) # complex(4.8, -1.5)
imag_part = float_complex.imag
print(f"7. Imaginary part of {float_complex}: {imag_part}") # Output:
-1.5

# 8. Imaginary part from a calculation result
c1 = 1+2j
c2 = 3+4j
sum_c = c1 + c2 # 4+6j
imag_part = sum_c.imag
print(f"8. Imaginary part of sum ({sum_c}): {imag_part}") # Output:
6.0

# 9. Imaginary part of conjugate
conjugate = growth_projection.conjugate() # 10-3j
imag_part = conjugate.imag
print(f"9. Imaginary part of conjugate ({conjugate}): {imag_part}") #
Output: -3.0 (negative of original)

# 10. Imaginary part when imag is zero
c = 7+0j
imag_part = c.imag
print(f"10. Imaginary part of {c}: {imag_part}") # Output: 0.0
```

d) `complex.conjugate()`

Explanation: Method called on a complex number object. Returns the complex conjugate, which has the same real part but an imaginary part with the opposite sign. If `z = a + bj`, its conjugate is `a - bj`. **Examples:**

```
growth_projection = 10+3j
# 1. Conjugate of growth_projection
conj = growth_projection.conjugate()
print(f"1. Conjugate of {growth_projection}: {conj}") # Output:
(10-3j)

# 2. Conjugate of a pure imaginary number
imag_num = 5j
conj = imag_num.conjugate()
```

```
print(f"2. Conjugate of {imag_num}: {conj}") # Output: (-0-5j) or -5j

# 3. Conjugate of a real number (no change)
real_num = complex(total_students, 0) # 1500+0j
conj = real_num.conjugate()
print(f"3. Conjugate of {real_num}: {conj}") # Output: (1500+0j)

# 4. Conjugate of a number with negative imaginary part
neg_imag_complex = 4-7j
conj = neg_imag_complex.conjugate()
print(f"4. Conjugate of {neg_imag_complex}: {conj}") # Output: (4+7j)

# 5. Conjugate of zero
zero_complex = 0j
conj = zero_complex.conjugate()
print(f"5. Conjugate of {zero_complex}: {conj}") # Output: 0j

# 6. Conjugate of a conjugate (returns original)
conj1 = growth_projection.conjugate() # 10-3j
conj2 = conj1.conjugate() # 10+3j
print(f"6. Conjugate of conjugate of {growth_projection}: {conj2}") #
Output: (10+3j)

# 7. Conjugate with float components
float_complex = complex(4.8, -1.5)
conj = float_complex.conjugate()
print(f"7. Conjugate of {float_complex}: {conj}") # Output:
(4.8+1.5j)

# 8. Conjugate of a sum: conj(c1+c2) == conj(c1) + conj(c2)
c1 = 1+2j
c2 = 3-4j
sum_conj = (c1 + c2).conjugate() # (4-2j).conjugate() -> 4+2j
ind_conj_sum = c1.conjugate() + c2.conjugate() # (1-2j) + (3+4j) ->
4+2j
print(f"8. Conjugate of sum: {sum_conj}, Sum of conjugates:
{ind_conj_sum}") # Output: (4+2j), (4+2j)

# 9. Conjugate of a product: conj(c1*c2) == conj(c1) * conj(c2)
prod_conj = (c1 * c2).conjugate() # ((1+2j)*(3-4j)).conjugate() =
(11+2j).conjugate() = 11-2j
ind_conj_prod = c1.conjugate() * c2.conjugate() # (1-2j)*(3+4j) =
11-2j
print(f"9. Conjugate of product: {prod_conj}, Product of conjugates:
{ind_conj_prod}") # Output: (11-2j), (11-2j)

# 10. Conjugate of a negative real part number
neg_real_complex = -5+2j
conj = neg_real_complex.conjugate()
print(f"10. Conjugate of {neg_real_complex}: {conj}") # Output:
```

```
(-5-2j)
```

e) `abs()` with Complex Numbers

Explanation: When applied to a complex number `z = a + bj`, `abs(z)` calculates its magnitude (or modulus), which is the distance from the origin (0,0) to the point (a,b) in the complex plane. It's calculated as `sqrt(a^2 + b^2)` and the result is a `float`. **Examples:**

```
growth_projection = 10+3j # a=10, b=3
# 1. Magnitude of growth_projection
magnitude = abs(growth_projection) # sqrt(10^2 + 3^2) = sqrt(100 + 9)
= sqrt(109) approx 10.44
print(f"1. Magnitude (abs) of {growth_projection}: {magnitude:.4f}")
# Output: 10.4403

# 2. Magnitude of a pure imaginary number
imag_num = 5j # a=0, b=5
magnitude = abs(imag_num) # sqrt(0^2 + 5^2) = sqrt(25) = 5.0
print(f"2. Magnitude of {imag_num}: {magnitude}") # Output: 5.0

# 3. Magnitude of a real number (absolute value)
real_num = complex(-1500, 0) # a=-1500, b=0
magnitude = abs(real_num) # sqrt((-1500)^2 + 0^2) = sqrt(1500^2) =
1500.0
print(f"3. Magnitude of {real_num}: {magnitude}") # Output: 1500.0

# 4. Magnitude of 3+4j (common Pythagorean triple)
c = 3+4j # a=3, b=4
magnitude = abs(c) # sqrt(3^2 + 4^2) = sqrt(9 + 16) = sqrt(25) = 5.0
print(f"4. Magnitude of {c}: {magnitude}") # Output: 5.0

# 5. Magnitude of zero
zero_complex = 0j
magnitude = abs(zero_complex) # sqrt(0^2 + 0^2) = 0.0
print(f"5. Magnitude of {zero_complex}: {magnitude}") # Output: 0.0

# 6. Magnitude of a complex number and its conjugate are equal
conj = growth_projection.conjugate() # 10-3j
mag_conj = abs(conj) # sqrt(10^2 + (-3)^2) = sqrt(100 + 9) =
sqrt(109)
print(f"6. Magnitude of conjugate {conj}: {mag_conj:.4f}") # Output:
10.4403

# 7. Magnitude involving floats
float_complex = complex(1.5, -2.0) # a=1.5, b=-2.0
magnitude = abs(float_complex) # sqrt(1.5^2 + (-2.0)^2) = sqrt(2.25 +
4) = sqrt(6.25) = 2.5
print(f"7. Magnitude of {float_complex}: {magnitude}") # Output: 2.5
```

```
# 8. abs(c1*c2) == abs(c1) * abs(c2)
c1 = 1+1j # abs(c1) = sqrt(1^2+1^2) = sqrt(2)
c2 = 2+3j # abs(c2) = sqrt(2^2+3^2) = sqrt(13)
prod = c1 * c2 # (1+1j)(2+3j) = (2-3) + (3+2)j = -1+5j
abs_prod = abs(prod) # sqrt((-1)^2 + 5^2) = sqrt(1+25) = sqrt(26)
abs1_abs2 = abs(c1) * abs(c2) # sqrt(2) * sqrt(13) = sqrt(26)
print(f"8. |product|: {abs_prod:.4f}, |c1|*|c2|: {abs1_abs2:.4f}") #
Output: 5.0990, 5.0990

# 9. Magnitude of a complex number with only negative parts
neg_complex = -2-7j
magnitude = abs(neg_complex) # sqrt((-2)^2 + (-7)^2) = sqrt(4 + 49) =
sqrt(53)
print(f"9. Magnitude of {neg_complex}: {magnitude:.4f}") # Output:
7.2801

# 10. Magnitude is always non-negative
print(f"10. Is magnitude of {growth_projection} >= 0?
{abs(growth_projection) >= 0}") # Output: True
```

f) `complex()`

Explanation: The type constructor. Can be used to create complex numbers from real and imaginary parts (which can be int or float), or by parsing a string representation. **Examples:**

```
# 1. From two integers (like the initial growth_projection)
c = complex(10, 3)
print(f"1. complex(10, 3) = {c}") # Output: (10+3j)

# 2. From two floats
c = complex(average_rating, -2.5) # complex(4.8, -2.5)
print(f"2. complex({average_rating}, -2.5) = {c}") # Output:
(4.8-2.5j)

# 3. From an integer (imaginary part defaults to 0)
c = complex(total_students) # complex(1500)
print(f"3. complex({total_students}) = {c}") # Output: (1500+0j)

# 4. From a float (imaginary part defaults to 0)
c = complex(average_rating) # complex(4.8)
print(f"4. complex({average_rating}) = {c}") # Output: (4.8+0j)

# 5. From a string representation (no spaces around +/-)
c = complex("10+3j")
print(f"5. complex('10+3j') = {c}") # Output: (10+3j)
c_neg = complex("5-2j")
print(f"   complex('5-2j') = {c_neg}") # Output: (5-2j)
```

```
c_imag = complex("-7j")
print(f"   complex('-7j') = {c_imag}") # Output: -7j

# 6. From a string with floats
c = complex("4.8-2.5j")
print(f"6. complex('4.8-2.5j') = {c}") # Output: (4.8-2.5j)

# 7. Using keyword arguments
c = complex(real=10, imag=3)
print(f"7. complex(real=10, imag=3) = {c}") # Output: (10+3j)

# 8. Creating zero complex number
c = complex(0, 0)
print(f"8. complex(0, 0) = {c}") # Output: 0j
c_str = complex("0j")
print(f"   complex('0j') = {c_str}") # Output: 0j

# 9. Creating pure imaginary number
c = complex(imag=5) # real defaults to 0
print(f"9. complex(imag=5) = {c}") # Output: 5j

# 10. Attempting invalid string conversion (raises ValueError)
try:
    invalid_complex = complex("10 + 3j") # Space is invalid here
except ValueError as e:
    print(f"10. Error converting '10 + 3j': {e}") # Output: Error ...
complex() arg is a malformed string
```

This covers the `int`, `float`, and `complex` numeric types in detail, along with explanations and 10 examples for their common functions, methods, and operations, incorporating the Edukron context where feasible.

Okay, let's continue with the remaining Python built-in data types, starting with the **Text Type (`str`)**.

🧵 Text Type (`str`)

Explanation: Strings (`str`) are used to represent textual data. They are sequences of Unicode characters, enclosed in single quotes (`'...'`), double quotes (`"..."`), or triple quotes (`'''...'''` or `"""..."""`). Triple quotes are often used for multi-line strings or docstrings. Strings are **immutable**, meaning once a string is created, its contents cannot be changed directly. Operations that seem to modify strings actually create and return *new* strings.

Your examples:

- `institute = "Edukron"`
- `focus_area = "Data Science"`
- `tech_stack = "Azure DevOps and Python"`
- `course_slogan = "Learn Data Engineering with Real-Time Projects"`

Common Operations & Methods:

Strings have a large number of built-in methods for manipulation and querying.

a) Concatenation (`+`) and Repetition (`*`)

Explanation:

- `+`: Joins two strings together, creating a new string.
- `*`: Repeats a string a specified number of times (using an integer), creating a new string. **Examples:**

```
institute = "Edukron"
focus_area = "Data Science"

# 1. Combining institute and focus area
full_title = institute + ": " + focus_area
print(f"1. Full title: {full_title}") # Output: Edukron: Data Science

# 2. Creating a course welcome message
course_name = "Azure DevOps"
welcome_message = "Welcome to " + institute + "'s " + course_name + "
course!"
print(f"2. Welcome message: {welcome_message}") # Output: Welcome to
Edukron's Azure DevOps course!

# 3. Repeating a separator
separator = "-*=" * 10
print(f"3. Separator: {separator}") # Output: -*=-*=-*=-*=-*=-*=-*=-*=-
*=-*=

# 4. Emphasizing a keyword
keyword = " Python "
emphasized = keyword * 3
print(f"4. Emphasized keyword: {emphasized}") # Output:  Python
Python  Python

# 5. Building a file path (simple example, os.path.join is better)
base_path = "/courses/"
course_id = "DA101"
```

```python
full_path = base_path + course_id
print(f"5. Simple path: {full_path}") # Output: /courses/DA101

# 6. Adding a suffix
student_name = "Alex"
username_suffix = "@edukron.com"
email = student_name + username_suffix
print(f"6. Student email: {email}") # Output: Alex@edukron.com

# 7. Repeating a character for underlining (approximate length)
heading = "Course Modules"
underline = "=" * len(heading) # len() gives length
print(f"7. Heading:\n{heading}\n{underline}")
# Output:
# Course Modules
# ==============

# 8. Concatenating multiple strings
part1 = "Learn "
part2 = "Data Engineering "
part3 = "Now!"
call_to_action = part1 + part2 + part3
print(f"8. Call to action: {call_to_action}") # Output: Learn Data
Engineering Now!

# 9. Repeating an empty string (results in empty string)
empty_repeated = "" * 100
print(f"9. Repeating empty string: '{empty_repeated}'") # Output: ''

# 10. Repeating a string zero times (results in empty string)
message = "ALERT! "
no_alerts = message * 0
print(f"10. Repeating zero times: '{no_alerts}'") # Output: ''
```

b) `len()`

Explanation: A built-in function (not a method) that returns the number of characters (length) of a string. **Examples:**

```python
institute = "Edukron"
course_slogan = "Learn Data Engineering with Real-Time Projects"
tech_stack = "Azure DevOps and Python"

# 1. Length of the institute name
name_length = len(institute)
print(f"1. Length of '{institute}': {name_length}") # Output: 7

# 2. Length of the course slogan
slogan_length = len(course_slogan)
```

```python
print(f"2. Length of slogan: {slogan_length}") # Output: 45

# 3. Length of the tech stack description
stack_length = len(tech_stack)
print(f"3. Length of tech stack: {stack_length}") # Output: 24

# 4. Length of an empty string
empty_len = len("")
print(f"4. Length of empty string: {empty_len}") # Output: 0

# 5. Length of a string with only spaces
spaces = "   "
spaces_len = len(spaces)
print(f"5. Length of '{spaces}': {spaces_len}") # Output: 3

# 6. Using len() for validation (e.g., minimum password length)
password = "pass"
min_len = 8
if len(password) < min_len:
    print(f"6. Password '{password}' is too short (min {min_len}
chars).") # Output: Password 'pass' is too short (min 8 chars).

# 7. Length of a string containing numbers and symbols
complex_str = "Batch #101!"
complex_len = len(complex_str)
print(f"7. Length of '{complex_str}': {complex_len}") # Output: 11

# 8. Length after concatenation
combined = institute + tech_stack # "EdukronAzure DevOps and Python"
combined_len = len(combined)
print(f"8. Length of combined string: {combined_len}") # Output: 31
(7 + 24)

# 9. Length of a single character string
char_str = "A"
char_len = len(char_str)
print(f"9. Length of '{char_str}': {char_len}") # Output: 1

# 10. Length of a multi-line string (includes newline characters)
multi_line = """Line 1
Line 2"""
multi_len = len(multi_line)
print(f"10. Length of multi-line string:\n'''{multi_line}'''\nis:
{multi_len}")
# Output:
# Length of multi-line string:
# '''Line 1
# Line 2'''
# is: 13 (Line 1 is 6, \n is 1, Line 2 is 6 => 6+1+6=13)
```

c) Indexing (`[]`) and Slicing (`[:]`)

Explanation:

- **Indexing:** Accessing a single character at a specific position (index). Python uses 0-based indexing (first character is at index 0). Negative indices count from the end (-1 is the last character).
- **Slicing:** Extracting a portion (substring) of the string using `[start:stop:step]`.
    - `start` : The index where the slice begins (inclusive). Defaults to 0.
    - `stop` : The index where the slice ends (exclusive). Defaults to the end of the string.
    - `step` : The increment between indices. Defaults to 1.

Examples:

```
institute = "Edukron"
tech_stack = "Azure DevOps and Python"

# --- Indexing ---
# 1. Get the first character of institute
first_char = institute[0]
print(f"1. First character of '{institute}': {first_char}") # Output:
E

# 2. Get the last character of institute
last_char = institute[-1]
print(f"2. Last character of '{institute}': {last_char}") # Output: n

# 3. Get the character at index 3
char_at_3 = institute[3]
print(f"3. Character at index 3 of '{institute}': {char_at_3}") #
Output: k

# --- Slicing ---
# 4. Get the first 5 characters of tech_stack ("Azure")
first_5 = tech_stack[0:5] # or [:5]
print(f"4. First 5 chars of tech_stack: '{first_5}'") # Output:
'Azure'

# 5. Get the substring "DevOps" (index 6 to 12, exclusive)
devops_part = tech_stack[6:12]
print(f"5. Extracting 'DevOps': '{devops_part}'") # Output: 'DevOps'

# 6. Get the last 6 characters ("Python")
last_6 = tech_stack[-6:]
```

```
print(f"6. Last 6 chars of tech_stack: '{last_6}'") # Output:
'Python'

# 7. Get everything *except* the first and last character
middle = institute[1:-1]
print(f"7. Middle part of '{institute}': '{middle}'") # Output:
'dukro'

# 8. Get every second character of institute
every_second = institute[::2]
print(f"8. Every second char of '{institute}': '{every_second}'") #
Output: 'Eurn'

# 9. Reverse the institute name using slicing
reversed_name = institute[::-1]
print(f"9. Reversed '{institute}': '{reversed_name}'") # Output:
'norkudE'

# 10. Get a slice from index 6 onwards
from_index_6 = tech_stack[6:]
print(f"10. Tech stack from index 6: '{from_index_6}'") # Output:
'DevOps and Python'
```

d) `str.lower()` and `str.upper()`

Explanation:

- `lower()`: Returns a *new* string with all characters converted to lowercase.
- `upper()`: Returns a *new* string with all characters converted to uppercase. Useful for case-insensitive comparisons. **Examples:**

```
institute = "Edukron"
focus_area = "Data Science"
tech_stack = "Azure DevOps and Python"

# 1. Institute name in lowercase
lower_institute = institute.lower()
print(f"1. Lowercase institute: '{lower_institute}'") # Output:
'edukron'

# 2. Focus area in uppercase
upper_focus = focus_area.upper()
print(f"2. Uppercase focus area: '{upper_focus}'") # Output: 'DATA
SCIENCE'

# 3. Tech stack in lowercase
lower_stack = tech_stack.lower()
print(f"3. Lowercase tech stack: '{lower_stack}'") # Output: 'azure
```

```
devops and python'

# 4. Case-insensitive comparison
user_input = "data science"
if user_input.lower() == focus_area.lower():
    print("4. User input matches focus area (case-insensitive).") #
Output: Matches

# 5. Converting mixed case to uppercase
mixed_case = "PyThOn PrOjEcT"
upper_mixed = mixed_case.upper()
print(f"5. Uppercase mixed case: '{upper_mixed}'") # Output: 'PYTHON
PROJECT'

# 6. Converting mixed case to lowercase
lower_mixed = mixed_case.lower()
print(f"6. Lowercase mixed case: '{lower_mixed}'") # Output: 'python
project'

# 7. Applying to a string with numbers/symbols (they remain
unchanged)
code = "Batch-101-DS"
lower_code = code.lower()
upper_code = code.upper()
print(f"7. Lowercase code: '{lower_code}', Uppercase code:
'{upper_code}'")
# Output: 'batch-101-ds', 'BATCH-101-DS'

# 8. Chaining after concatenation
full_title = institute + ": " + focus_area
upper_title = full_title.upper()
print(f"8. Uppercase full title: '{upper_title}'") # Output:
'EDUKRON: DATA SCIENCE'

# 9. Applying to an already lowercase string
already_lower = "real-time"
result = already_lower.lower()
print(f"9. Applying lower() to lowercase string: '{result}'") #
Output: 'real-time'

# 10. Applying to an already uppercase string
already_upper = "AZURE"
result = already_upper.upper()
print(f"10. Applying upper() to uppercase string: '{result}'") #
Output: 'AZURE'
```

e) `str.strip()`, `str.lstrip()`, `str.rstrip()`

Explanation:

- `strip([chars])`: Returns a *new* string with leading and trailing characters removed. If `chars` is omitted or `None`, it removes whitespace characters. If `chars` is given, it removes any character in the `chars` string from the ends.
- `lstrip([chars])`: Removes leading characters only.
- `rstrip([chars])`: Removes trailing characters only. Useful for cleaning up user input or data read from files. **Examples:**

```
# 1. Removing leading/trailing whitespace
user_input = "  Data Science  "
cleaned_input = user_input.strip()
print(f"1. Stripped input: '{cleaned_input}'") # Output: 'Data
Science'

# 2. Removing only leading whitespace
leading_space = "   Edukron"
left_stripped = leading_space.lstrip()
print(f"2. Left-stripped: '{left_stripped}'") # Output: 'Edukron'

# 3. Removing only trailing whitespace
trailing_space = "Python   "
right_stripped = trailing_space.rstrip()
print(f"3. Right-stripped: '{right_stripped}'") # Output: 'Python'

# 4. Stripping specific characters (e.g., punctuation)
course_code = "===DS101==="
cleaned_code = course_code.strip("=")
print(f"4. Stripped specific chars: '{cleaned_code}'") # Output:
'DS101'

# 5. Stripping multiple different characters from ends
path = "/courses/Azure/"
cleaned_path = path.strip("/e") # Removes '/' and 'e' from ends
print(f"5. Stripping multiple chars: '{cleaned_path}'") # Output:
'courses/Azur'

# 6. Using lstrip with specific characters
filename = "__init__.py"
stripped_filename = filename.lstrip("_")
print(f"6. Left-stripped specific chars: '{stripped_filename}'") #
Output: 'init__.py'

# 7. Using rstrip with specific characters
version = "v1.0.RELEASE..."
stripped_version = version.rstrip(".RELEASE") # Removes '.', 'R',
'E', 'L', 'A', 'S' from end
print(f"7. Right-stripped specific chars: '{stripped_version}'") #
Output: 'v1.0'
```

```python
# 8. Stripping whitespace from a multi-line string's ends
multi_line = "\n  Line 1 \n Line 2 \t\n"
stripped_multi = multi_line.strip()
print(f"8. Stripped multi-line:\n'''{stripped_multi}''''")
# Output:
# '''Line 1
# Line 2'''

# 9. No characters to strip
clean_string = "Edukron"
result = clean_string.strip()
print(f"9. Stripping a clean string: '{result}'") # Output: 'Edukron'

# 10. Stripping only characters specified
special_format = "***Python***"
# Only strips '*' not whitespace if specified
result = special_format.strip('*')
print(f"10. Stripping only '*': '{result}'") # Output: 'Python'
```

f) `str.replace(old, new[, count])`

Explanation: Returns a *new* string where all occurrences of the substring `old` are replaced with the substring `new`. If the optional argument `count` is given, only the first `count` occurrences are replaced. **Examples:**

```python
course_slogan = "Learn Data Engineering with Real-Time Projects"
tech_stack = "Azure DevOps and Python and SQL"

# 1. Replacing 'Data Engineering' with 'Data Science'
new_slogan = course_slogan.replace("Data Engineering", "Data
Science")
print(f"1. New slogan: '{new_slogan}'")
# Output: 'Learn Data Science with Real-Time Projects'

# 2. Replacing spaces with underscores
slug = course_slogan.replace(" ", "_").lower() # Chain with lower()
print(f"2. Slugified slogan: '{slug}'")
# Output: 'learn_data_engineering_with_real-time_projects'

# 3. Replacing 'and' with '&' in tech_stack
ampersand_stack = tech_stack.replace("and", "&")
print(f"3. Tech stack with ampersand: '{ampersand_stack}'")
# Output: 'Azure DevOps & Python & SQL'

# 4. Replacing only the first occurrence of 'and'
first_ampersand = tech_stack.replace("and", "&", 1)
print(f"4. Replacing first 'and' only: '{first_ampersand}'")
# Output: 'Azure DevOps & Python and SQL'
```

```
# 5. Removing all spaces
no_spaces = course_slogan.replace(" ", "")
print(f"5. Slogan without spaces: '{no_spaces}'")
# Output: 'LearnDataEngineeringwithReal-TimeProjects'

# 6. Replacing a character that doesn't exist (no change)
result = course_slogan.replace("Java", "Python")
print(f"6. Replacing non-existent substring: '{result}'")
# Output: 'Learn Data Engineering with Real-Time Projects'

# 7. Replacing with an empty string (effectively deleting)
deleted_realtime = course_slogan.replace("Real-Time ", "")
print(f"7. Deleting 'Real-Time ': '{deleted_realtime}'")
# Output: 'Learn Data Engineering with Projects'

# 8. Case-sensitive replacement
text = "Python python pYthon"
result = text.replace("python", "PYTHON") # Only replaces lowercase
'python'
print(f"8. Case-sensitive replace: '{result}'")
# Output: 'Python PYTHON pYthon'

# 9. Replacing multiple characters (by chaining)
filename = "project report final.docx"
clean_filename = filename.replace(" ", "_").replace(".docx", ".pdf")
print(f"9. Chained replace for filename: '{clean_filename}'")
# Output: 'project_report_final.pdf'

# 10. Replacing using count=0 (no change)
result = tech_stack.replace("and", "&", 0)
print(f"10. Replacing with count=0: '{result}'")
# Output: 'Azure DevOps and Python and SQL'
```

g) `str.split(sep=None, maxsplit=-1)`

Explanation: Returns a *list* of substrings (words) by splitting the string at occurrences of the separator `sep`.

- If `sep` is omitted or `None`, splits by any whitespace character and discards empty strings.
- If `sep` is specified, splits exactly at that separator.
- `maxsplit`: If specified, performs at most `maxsplit` splits (resulting in `maxsplit + 1` elements in the list). The remainder of the string is the last element. **Examples:**

```
course_slogan = "Learn Data Engineering with Real-Time Projects"
tech_stack = "Azure DevOps,Python,SQL,Git"
```

```python
institute = "Edukron"

# 1. Splitting slogan by whitespace (default)
words = course_slogan.split()
print(f"1. Slogan words: {words}")
# Output: ['Learn', 'Data', 'Engineering', 'with', 'Real-Time',
'Projects']

# 2. Splitting tech stack by comma
skills = tech_stack.split(',')
print(f"2. Tech skills: {skills}")
# Output: ['Azure DevOps', 'Python', 'SQL', 'Git']

# 3. Splitting institute name by character 'k'
parts = institute.split('k')
print(f"3. Splitting '{institute}' by 'k': {parts}") # Output:
['Edu', 'ron']

# 4. Splitting slogan with maxsplit=2
split_limit = course_slogan.split(maxsplit=2)
print(f"4. Splitting slogan (maxsplit=2): {split_limit}")
# Output: ['Learn', 'Data', 'Engineering with Real-Time Projects']

# 5. Splitting tech stack by comma with maxsplit=1
split_tech_limit = tech_stack.split(',', maxsplit=1)
print(f"5. Splitting tech stack (maxsplit=1): {split_tech_limit}")
# Output: ['Azure DevOps', 'Python,SQL,Git']

# 6. Splitting a string with multiple spaces (default handles it)
multi_space = "Azure   DevOps  Python"
words = multi_space.split()
print(f"6. Splitting multi-space string: {words}")
# Output: ['Azure', 'DevOps', 'Python']

# 7. Splitting by a separator that is not present
result = course_slogan.split("---")
print(f"7. Splitting by non-existent separator: {result}")
# Output: ['Learn Data Engineering with Real-Time Projects'] (list
with 1 element)

# 8. Splitting an empty string
result = "".split()
print(f"8. Splitting empty string (default sep): {result}") # Output:
[]
result_sep = "".split(',')
print(f"   Splitting empty string (with sep): {result_sep}") #
Output: ['']

# 9. Splitting a string that starts/ends with separator (using sep)
path = "/data/science/"
```

```
parts = path.split('/')
print(f"9. Splitting path '{path}' by '/': {parts}")
# Output: ['', 'data', 'science', ''] (Note the empty strings)

# 10. Splitting line breaks
multi_line = "Edukron\nData Science\nAzure"
lines = multi_line.split('\n')
print(f"10. Splitting by newline: {lines}")
# Output: ['Edukron', 'Data Science', 'Azure']
```

h) `str.join(iterable)`

Explanation: The opposite of `split`. Takes an iterable (like a list or tuple) of strings and concatenates its elements, using the string *on which the method is called* as the separator between elements. **Examples:**

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
tech_skills = ("Python", "SQL", "Git", "CI/CD")
words = ['Learn', 'Data', 'Engineering']

# 1. Joining course list with a comma and space
joined_courses = ", ".join(course_list)
print(f"1. Joined courses: '{joined_courses}'")
# Output: 'Data Science, Data Engineering, Azure DevOps'

# 2. Joining tech skills with a hyphen
joined_skills = "-".join(tech_skills)
print(f"2. Joined skills: '{joined_skills}'")
# Output: 'Python-SQL-Git-CI/CD'

# 3. Joining words back into a sentence with spaces
sentence = " ".join(words)
print(f"3. Rejoined sentence: '{sentence}'") # Output: 'Learn Data
Engineering'

# 4. Joining with an empty string (no separator)
institute = "Edukron"
chars = list(institute) # ['E', 'd', 'u', 'k', 'r', 'o', 'n']
joined_chars = "".join(chars)
print(f"4. Joining chars with empty string: '{joined_chars}'") #
Output: 'Edukron'

# 5. Joining using a multi-character separator
separator = " | "
joined_fancy = separator.join(course_list)
print(f"5. Joining with ' | ': '{joined_fancy}'")
# Output: 'Data Science | Data Engineering | Azure DevOps'

# 6. Joining an iterable with only one element
```

```
single_item_list = ["Azure DevOps"]
result = ", ".join(single_item_list)
print(f"6. Joining single item list: '{result}'") # Output: 'Azure
DevOps'

# 7. Joining an empty iterable
empty_list = []
result = ", ".join(empty_list)
print(f"7. Joining empty list: '{result}'") # Output: ''

# 8. Joining elements that are not just words
items = ["Course", "Batch101", "Module5"]
identifier = "_".join(items)
print(f"8. Joining items with underscore: '{identifier}'")
# Output: 'Course_Batch101_Module5'

# 9. Joining using newline as separator
report_lines = ["Edukron Status Report", "Students: 1500", "Rating:
4.8"]
report = "\n".join(report_lines)
print(f"9. Joining with newline:\n{report}")
# Output:
# Edukron Status Report
# Students: 1500
# Rating: 4.8

# 10. Joining requires all elements to be strings (TypeError
otherwise)
mixed_list = ["Data Science", 101]
try:
    error_join = ", ".join(mixed_list)
except TypeError as e:
    print(f"10. Error joining list with non-string: {e}")
# Output: Error joining list with non-string: sequence item 1:
expected str instance, int found
```

i) `str.startswith(prefix[, start[, end]])` and `str.endswith(suffix[, start[, end]])`

Explanation:

- `startswith()`: Returns `True` if the string starts with the specified `prefix`, `False` otherwise.
- `endswith()`: Returns `True` if the string ends with the specified `suffix`, `False` otherwise.
- Optional `start` and `end` arguments restrict the check to a slice of the string.

- `prefix` / `suffix` can also be a tuple of strings to check against.

  Examples:

```
institute = "Edukron"
course_slogan = "Learn Data Engineering with Real-Time Projects"
filename = "edukron_report.pdf"

# 1. Check if institute starts with "Edu"
starts_edu = institute.startswith("Edu")
print(f"1. Does '{institute}' start with 'Edu'? {starts_edu}") #
Output: True

# 2. Check if institute starts with "Data"
starts_data = institute.startswith("Data")
print(f"2. Does '{institute}' start with 'Data'? {starts_data}") #
Output: False

# 3. Check if slogan ends with "Projects"
ends_projects = course_slogan.endswith("Projects")
print(f"3. Does slogan end with 'Projects'? {ends_projects}") #
Output: True

# 4. Check if filename ends with ".pdf"
ends_pdf = filename.endswith(".pdf")
print(f"4. Does '{filename}' end with '.pdf'? {ends_pdf}") # Output:
True

# 5. Check if filename ends with ".csv" or ".pdf"
ends_data_format = filename.endswith((".csv", ".pdf"))
print(f"5. Does '{filename}' end with '.csv' or '.pdf'?
{ends_data_format}") # Output: True

# 6. Check if slogan starts with "Learn" or "Explore"
starts_learn_explore = course_slogan.startswith(("Learn", "Explore"))
print(f"6. Does slogan start with 'Learn' or 'Explore'?
{starts_learn_explore}") # Output: True

# 7. Check a slice: Does "Engineering" start at index 11 in slogan?
slice_check_start = course_slogan.startswith("Engineering", 11)
print(f"7. Does slogan start with 'Engineering' at index 11?
{slice_check_start}") # Output: True

# 8. Check a slice: Does "Real-Time" end at index 36 in slogan?
# Slice [start:end] includes start, excludes end. So we check up to
index 36.
slice_check_end = course_slogan.endswith("Real-Time", 0, 36)
print(f"8. Does slogan[0:36] end with 'Real-Time'?
{slice_check_end}") # Output: True
```

```
# 9. Case-sensitive check (starts with 'learn')
starts_learn_lower = course_slogan.startswith("learn")
print(f"9. Does slogan start with 'learn' (lowercase)? {starts_learn_lower}") # Output: False

# 10. Check endswith on an empty string
ends_anything = "".endswith("anything")
print(f"10. Does '' end with 'anything'? {ends_anything}") # Output: False (but True if suffix is also "")
```

j) `str.find(sub[, start[, end]])` and `str.index(sub[, start[, end]])`

Explanation:

- `find()`: Searches for the first occurrence of substring `sub` within the string (or the specified slice). Returns the starting index if found, and **-1** if not found.
- `index()`: Same as `find()`, but raises a **ValueError** if the substring is not found. Use `find()` if you expect the substring might be missing and want to handle it gracefully (check for -1). Use `index()` if you expect the substring to be present and want an error if it's not.

  Examples:

```
course_slogan = "Learn Data Engineering with Real-Time Projects"
tech_stack = "Azure DevOps and Python"

# 1. Find the index of "Data" in the slogan
data_pos = course_slogan.find("Data")
print(f"1. Index of 'Data' using find(): {data_pos}") # Output: 6

# 2. Find the index of "Python" in tech_stack
python_pos = tech_stack.find("Python")
print(f"2. Index of 'Python' using find(): {python_pos}") # Output: 18

# 3. Find a substring that doesn't exist
java_pos = course_slogan.find("Java")
print(f"3. Index of 'Java' using find(): {java_pos}") # Output: -1

# 4. Use index() to find "Azure" (will succeed)
azure_pos = tech_stack.index("Azure")
print(f"4. Index of 'Azure' using index(): {azure_pos}") # Output: 0

# 5. Use index() to find "Java" (will fail)
try:
    java_pos_index = course_slogan.index("Java")
except ValueError as e:
```

```
    print(f"5. Error using index() to find 'Java': {e}")
# Output: Error using index() to find 'Java': substring not found

# 6. Find "Engineering" starting search from index 10
eng_pos_slice = course_slogan.find("Engineering", 10)
print(f"6. Index of 'Engineering' starting at 10: {eng_pos_slice}") #
Output: 11

# 7. Find "e" starting search from index 2
e_pos_slice = course_slogan.find("e", 2) # First 'e' is at index 1,
next is at 12
print(f"7. Index of 'e' starting at 2: {e_pos_slice}") # Output: 12

# 8. Find the last occurrence (using rfind() or rindex())
last_e_pos = course_slogan.rfind("e") # rfind searches from the right
print(f"8. Index of last 'e' using rfind(): {last_e_pos}") # Output:
43

# 9. Find using an end index: find 'Data' before index 10
data_pos_end = course_slogan.find("Data", 0, 10)
print(f"9. Index of 'Data' before index 10: {data_pos_end}") #
Output: 6

# 10. Find in an empty string
empty_find = "".find("a")
print(f"10. Find 'a' in empty string: {empty_find}") # Output: -1
```

This covers the `str` type. Next up are the Sequence Types: `list`, `tuple`, and `range`. Shall I proceed with `list`?

Okay, diving straight into the remaining data types without interruption.

📚 Sequence Types

Sequence types represent ordered collections of items. The order in which items are added is preserved, and items can be accessed by their numerical index (position).

1. `list`

Explanation: Lists are **mutable** (changeable), ordered sequences of items. They are defined using square brackets `[]`, with items separated by commas. Lists can contain items of different data types, making them very versatile.

Your example:

- `course_list = ["Data Science", "Data Engineering", "Azure DevOps"]`

Common Operations & Methods:

a) Creation and Indexing/Slicing

Explanation: Creating lists and accessing elements or sub-lists using indices (0-based) and slices (`[start:stop:step]`). **Examples:**

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps",
"Python Basics", "SQL Fundamentals"]
student_counts = [350, 420, 380, 500, 450]

# 1. Create a list of courses
print(f"1. Course List: {course_list}")

# 2. Access the first course
first_course = course_list[0]
print(f"2. First course: {first_course}") # Output: Data Science

# 3. Access the last course count
last_count = student_counts[-1]
print(f"3. Last course count: {last_count}") # Output: 450

# 4. Get a slice of the first three courses
first_three_courses = course_list[0:3] # or [:3]
print(f"4. First three courses: {first_three_courses}")
# Output: ['Data Science', 'Data Engineering', 'Azure DevOps']

# 5. Get a slice of student counts from index 1 to 3 (exclusive)
counts_slice = student_counts[1:3]
print(f"5. Student counts slice [1:3]: {counts_slice}") # Output:
[420, 380]

# 6. Get every second course starting from the first
every_second_course = course_list[::2]
print(f"6. Every second course: {every_second_course}")
# Output: ['Data Science', 'Azure DevOps', 'SQL Fundamentals']

# 7. Create a list with mixed data types
batch_info = ["DE101", "Data Engineering", 45, True] # ID, Name,
Size, Active
print(f"7. Mixed type list: {batch_info}")

# 8. Access an element in the mixed list
batch_size = batch_info[2]
print(f"8. Accessing batch size: {batch_size}") # Output: 45
```

```python
# 9. Get the last two items using slicing
last_two_items = batch_info[-2:]
print(f"9. Last two items: {last_two_items}") # Output: [45, True]

# 10. Create an empty list
upcoming_courses = []
print(f"10. Empty list: {upcoming_courses}")
```

b) `list.append(item)`

Explanation: Adds a single `item` to the *end* of the list. Modifies the list in-place.

Examples:

```python
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
new_enrollments = [15, 20, 10]

# 1. Add a new course to the list
course_list.append("Cloud Computing")
print(f"1. List after appending course: {course_list}")
# Output: ['Data Science', 'Data Engineering', 'Azure DevOps', 'Cloud
Computing']

# 2. Add another course
course_list.append("Machine Learning")
print(f"2. List after appending another: {course_list}")
# Output: ['Data Science', 'Data Engineering', 'Azure DevOps', 'Cloud
Computing', 'Machine Learning']

# 3. Append a number to a list of numbers
new_enrollments.append(18)
print(f"3. Enrollments after append: {new_enrollments}") # Output:
[15, 20, 10, 18]

# 4. Append a list as a single item (nested list)
meta_list = [1, 2]
meta_list.append([3, 4])
print(f"4. List after appending a list: {meta_list}") # Output: [1,
2, [3, 4]]

# 5. Append a student name to a batch list
batch_students = ["Alice", "Bob"]
batch_students.append("Charlie")
print(f"5. Batch students after append: {batch_students}") # Output:
['Alice', 'Bob', 'Charlie']

# 6. Start with an empty list and append items
feedback_scores = []
feedback_scores.append(5)
feedback_scores.append(4)
```

```python
feedback_scores.append(5)
print(f"6. Feedback scores built with append: {feedback_scores}") # Output: [5, 4, 5]

# 7. Append a boolean value
flags = [True, False]
flags.append(True)
print(f"7. Flags after append: {flags}") # Output: [True, False, True]

# 8. Append None
optional_features = ["Feature A"]
optional_features.append(None) # Representing an unconfigured feature
print(f"8. Features after appending None: {optional_features}") # Output: ['Feature A', None]

# 9. Append result of a calculation
values = [10, 20]
values.append(values[0] + values[1]) # Append 30
print(f"9. Values after appending sum: {values}") # Output: [10, 20, 30]

# 10. Append a tuple as a single item
coordinates = [(0,0), (1,1)]
coordinates.append((2, 3))
print(f"10. Coordinates after appending tuple: {coordinates}") # Output: [(0, 0), (1, 1), (2, 3)]
```

c) `list.extend(iterable)`

Explanation: Adds all items from an `iterable` (like another list, tuple, or string) to the end of the current list. Modifies the list in-place. Differs from `append` which adds the iterable *as a single element*. **Examples:**

```python
data_science_courses = ["DS Intro", "Python for DS"]
data_eng_courses = ["DE Intro", "SQL for DE", "ADF"]
new_tools = ("Docker", "Kubernetes")

# 1. Extend DS courses with DE courses
data_science_courses.extend(data_eng_courses)
print(f"1. Extended DS courses: {data_science_courses}")
# Output: ['DS Intro', 'Python for DS', 'DE Intro', 'SQL for DE', 'ADF']

# 2. Extend with a tuple of new tools
all_tools = ["Python", "SQL"]
all_tools.extend(new_tools)
print(f"2. All tools after extend: {all_tools}")
# Output: ['Python', 'SQL', 'Docker', 'Kubernetes']
```

```python
# 3. Extend with another list
batch_1_students = ["Alice", "Bob"]
batch_2_students = ["Charlie", "David"]
batch_1_students.extend(batch_2_students)
print(f"3. Combined batch 1 students: {batch_1_students}")
# Output: ['Alice', 'Bob', 'Charlie', 'David']

# 4. Extend with a string (adds each character)
letters = ['a', 'b']
letters.extend("cde")
print(f"4. Letters extended with string: {letters}") # Output: ['a',
'b', 'c', 'd', 'e']

# 5. Extend an empty list
empty_list = []
empty_list.extend([1, 2, 3])
print(f"5. Extending an empty list: {empty_list}") # Output: [1, 2,
3]

# 6. Extend with items from a set (order not guaranteed from set)
topics = ["Visualization"]
data_analytics_topics = {"Power BI", "EDA", "Reporting"} # Set
topics.extend(data_analytics_topics)
print(f"6. Topics extended with set items: {topics}") # Order of
added items might vary

# 7. Extend with a list created on the fly
numbers = [10, 20]
numbers.extend([30, 40])
print(f"7. Numbers extended with list literal: {numbers}") # Output:
[10, 20, 30, 40]

# 8. Extend using range (converted implicitly)
ids = [101, 102]
ids.extend(range(103, 105)) # Adds 103, 104
print(f"8. IDs extended with range: {ids}") # Output: [101, 102, 103,
104]

# 9. Extend a list with itself (doubles the list)
items = ["Module 1", "Module 2"]
items.extend(items)
print(f"9. Items extended with itself: {items}")
# Output: ['Module 1', 'Module 2', 'Module 1', 'Module 2']

# 10. Contrast with append: extending vs appending a list
list_a = [1, 2]
list_b = [3, 4]
list_a.extend(list_b) # list_a is now [1, 2, 3, 4]
print(f"10. After extend: {list_a}")
list_c = [1, 2]
```

```
list_c.append(list_b) # list_c is now [1, 2, [3, 4]]
print(f"    After append: {list_c}")
```

d) `list.insert(index, item)`

Explanation: Inserts an `item` at a specific `index`. Existing items from that index onwards are shifted to the right. Modifies the list in-place. **Examples:**

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
priority_tasks = ["Deploy App", "Monitor Logs"]

# 1. Insert "Python Basics" at the beginning (index 0)
course_list.insert(0, "Python Basics")
print(f"1. List after inserting at index 0: {course_list}")
# Output: ['Python Basics', 'Data Science', 'Data Engineering',
'Azure DevOps']

# 2. Insert "Cloud Intro" at index 2
course_list.insert(2, "Cloud Intro")
print(f"2. List after inserting at index 2: {course_list}")
# Output: ['Python Basics', 'Data Science', 'Cloud Intro', 'Data
Engineering', 'Azure DevOps']

# 3. Insert a high-priority task at the beginning
priority_tasks.insert(0, "Fix Critical Bug")
print(f"3. Priority tasks after insert: {priority_tasks}")
# Output: ['Fix Critical Bug', 'Deploy App', 'Monitor Logs']

# 4. Insert using a negative index (-1 inserts before the last item)
numbers = [1, 2, 3, 5]
numbers.insert(-1, 4) # Insert 4 before the last item (5)
print(f"4. Numbers after inserting at -1: {numbers}") # Output: [1,
2, 3, 4, 5]

# 5. Insert at an index equal to the length (equivalent to append)
tasks = ["Task A", "Task B"]
tasks.insert(len(tasks), "Task C")
print(f"5. Inserting at len(list): {tasks}") # Output: ['Task A',
'Task B', 'Task C']

# 6. Insert at an index larger than the length (inserts at the end)
modules = ["M1", "M2"]
modules.insert(100, "M3") # Index 100 is beyond length, inserts at
end
print(f"6. Inserting at index > len: {modules}") # Output: ['M1',
'M2', 'M3']

# 7. Insert at a negative index far beyond the beginning (inserts at
beginning)
flags = [True, False]
```

```
flags.insert(-100, None) # Index -100 is before beginning, inserts at
0
print(f"7. Inserting at index < -len: {flags}") # Output: [None,
True, False]

# 8. Insert a list as an item
nested = [1, [3, 4]]
nested.insert(1, 2)
print(f"8. Inserting into a list with nested list: {nested}") #
Output: [1, 2, [3, 4]]

# 9. Insert into an empty list (only index 0 is valid without append
behavior)
empty = []
empty.insert(0, "First Item")
print(f"9. Inserting into empty list: {empty}") # Output: ['First
Item']

# 10. Insert duplicate item
ratings = [4, 5, 5]
ratings.insert(1, 5)
print(f"10. Inserting a duplicate item: {ratings}") # Output: [4, 5,
5, 5]
```

e) `list.remove(item)`

Explanation: Removes the *first* occurrence of the specified `item` from the list. Raises a `ValueError` if the item is not found. Modifies the list in-place.

Examples:

```
course_list = ["Python Basics", "Data Science", "Cloud Intro", "Data
Engineering", "Data Science"]
tools = ["Python", "SQL", "Git", "SQL"]

# 1. Remove "Cloud Intro"
course_list.remove("Cloud Intro")
print(f"1. List after removing 'Cloud Intro': {course_list}")
# Output: ['Python Basics', 'Data Science', 'Data Engineering', 'Data
Science']

# 2. Remove the first occurrence of "Data Science"
course_list.remove("Data Science")
print(f"2. List after removing first 'Data Science': {course_list}")
# Output: ['Python Basics', 'Data Engineering', 'Data Science']

# 3. Remove the first occurrence of "SQL"
tools.remove("SQL")
print(f"3. Tools after removing first 'SQL': {tools}")
# Output: ['Python', 'Git', 'SQL']
```

```python
# 4. Remove "Git"
tools.remove("Git")
print(f"4. Tools after removing 'Git': {tools}")
# Output: ['Python', 'SQL']

# 5. Attempt to remove an item not in the list
try:
    tools.remove("Java")
except ValueError as e:
    print(f"5. Error removing 'Java': {e}")
# Output: Error removing 'Java': list.remove(x): x not in list

# 6. Remove an integer item
student_counts = [350, 420, 380, 420]
student_counts.remove(420) # Removes the first 420
print(f"6. Counts after removing first 420: {student_counts}") #
Output: [350, 380, 420]

# 7. Remove a boolean item
flags = [True, False, True]
flags.remove(True) # Removes the first True
print(f"7. Flags after removing first True: {flags}") # Output:
[False, True]

# 8. Remove None
items = ["A", None, "B", None]
items.remove(None) # Removes the first None
print(f"8. Items after removing first None: {items}") # Output: ['A',
'B', None]

# 9. Removing from a list with one item
single_item_list = ["Final Project"]
single_item_list.remove("Final Project")
print(f"9. Removing the only item: {single_item_list}") # Output: []

# 10. Remove item based on a variable
course_to_remove = "Data Engineering"
if course_to_remove in course_list:
    course_list.remove(course_to_remove)
print(f"10. List after removing '{course_to_remove}': {course_list}")
# Output: ['Python Basics', 'Data Science']
```

f) `list.pop([index])`

Explanation: Removes and *returns* the item at the given `index`. If `index` is omitted, it removes and returns the *last* item. Raises an `IndexError` if the list is empty or the index is out of range. Modifies the list in-place. **Examples:**

```python
course_list = ["Python Basics", "Data Science", "Data Engineering"]
tasks = ["Review Code", "Update Docs", "Deploy"]
```

```python
# 1. Pop the last item (default index -1)
last_course = course_list.pop()
print(f"1. Popped last course: '{last_course}', List is now:
{course_list}")
# Output: Popped last course: 'Data Engineering', List is now:
['Python Basics', 'Data Science']

# 2. Pop the item at index 0
first_course = course_list.pop(0)
print(f"2. Popped first course: '{first_course}', List is now:
{course_list}")
# Output: Popped first course: 'Python Basics', List is now: ['Data
Science']

# 3. Pop the last task
last_task = tasks.pop()
print(f"3. Popped last task: '{last_task}', Tasks left: {tasks}")
# Output: Popped last task: 'Deploy', Tasks left: ['Review Code',
'Update Docs']

# 4. Pop the item at index 1
middle_task = tasks.pop(1)
print(f"4. Popped task at index 1: '{middle_task}', Tasks left:
{tasks}")
# Output: Popped task at index 1: 'Update Docs', Tasks left: ['Review
Code']

# 5. Pop the only remaining item
only_task = tasks.pop()
print(f"5. Popped only task: '{only_task}', Tasks left: {tasks}")
# Output: Popped only task: 'Review Code', Tasks left: []

# 6. Attempt to pop from an empty list (causes IndexError)
try:
    tasks.pop()
except IndexError as e:
    print(f"6. Error popping from empty list: {e}")
# Output: Error popping from empty list: pop from empty list

# 7. Pop using a negative index (-1 is last, -2 is second last)
numbers = [10, 20, 30, 40]
second_last = numbers.pop(-2)
print(f"7. Popped second last number: {second_last}, List is now:
{numbers}")
# Output: Popped second last number: 30, List is now: [10, 20, 40]

# 8. Pop the new last number
new_last = numbers.pop()
print(f"8. Popped new last number: {new_last}, List is now:
```

```
{numbers}")
# Output: Popped new last number: 40, List is now: [10, 20]

# 9. Pop first element again
first_again = numbers.pop(0)
print(f"9. Popped first element again: {first_again}, List is now:
{numbers}")
# Output: Popped first element again: 10, List is now: [20]

# 10. Attempt to pop with index out of range
try:
    numbers.pop(5) # Only index 0 is valid now
except IndexError as e:
    print(f"10. Error popping index 5: {e}")
# Output: Error popping index 5: pop index out of range
```

g) `list.sort(key=None, reverse=False)`

Explanation: Sorts the items of the list *in-place*.

- `key`: A function to be called on each element before making comparisons (e.g., `key=str.lower` for case-insensitive sort).
- `reverse=True`: Sorts in descending order. Items must be comparable (e.g., all numbers or all strings). **Examples:**

```
course_list = ["Data Engineering", "Azure DevOps", "Data Science",
"Python Basics"]
student_counts = [350, 420, 380, 500, 450]
mixed_case_courses = ["azure DevOps", "Python basics", "Data
science"]

# 1. Sort course list alphabetically (in-place)
course_list.sort()
print(f"1. Sorted courses: {course_list}")
# Output: ['Azure DevOps', 'Data Engineering', 'Data Science',
'Python Basics']

# 2. Sort student counts numerically (ascending)
student_counts.sort()
print(f"2. Sorted counts: {student_counts}") # Output: [350, 380,
420, 450, 500]

# 3. Sort student counts in descending order
student_counts.sort(reverse=True)
print(f"3. Counts sorted descending: {student_counts}") # Output:
[500, 450, 420, 380, 350]

# 4. Sort mixed case courses case-insensitively
mixed_case_courses.sort(key=str.lower)
print(f"4. Mixed case sorted case-insensitively:
```

```
{mixed_case_courses}")
# Output: ['azure DevOps', 'Data science', 'Python basics']

# 5. Sort list of tuples based on the second element (student count)
course_enrollments = [("Data Science", 380), ("Python Basics", 500),
("Azure DevOps", 420)]
course_enrollments.sort(key=lambda item: item[1]) # Sort by count
(item[1])
print(f"5. Courses sorted by enrollment: {course_enrollments}")
# Output: [('Data Science', 380), ('Azure DevOps', 420), ('Python
Basics', 500)]

# 6. Sort descending by enrollment
course_enrollments.sort(key=lambda item: item[1], reverse=True)
print(f"6. Courses sorted descending by enrollment:
{course_enrollments}")
# Output: [('Python Basics', 500), ('Azure DevOps', 420), ('Data
Science', 380)]

# 7. Sort list based on length of strings
courses = ["SQL", "Python", "Azure DevOps", "Git"]
courses.sort(key=len)
print(f"7. Courses sorted by length: {courses}")
# Output: ['SQL', 'Git', 'Python', 'Azure DevOps']

# 8. Sort list of lists based on the first element
batches = [[102, 'DE'], [101, 'DS'], [103, 'ADO']]
batches.sort() # Default sorts by first element, then second if first
is equal
print(f"8. Batches sorted by ID: {batches}")
# Output: [[101, 'DS'], [102, 'DE'], [103, 'ADO']]

# 9. Sort a list that is already sorted
already_sorted = [10, 20, 30]
already_sorted.sort()
print(f"9. Sorting an already sorted list: {already_sorted}") #
Output: [10, 20, 30]

# 10. Attempt to sort list with incompatible types (TypeError)
incompatible_list = ["Data Science", 420, True]
try:
    incompatible_list.sort()
except TypeError as e:
    print(f"10. Error sorting incompatible types: {e}")
# Output: Error sorting incompatible types: '<' not supported between
instances of 'int' and 'str' (or similar)
```

h) `len(list)`

Explanation: Returns the number of items in the list. (Same built-in function as used for strings). **Examples:**

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
student_counts = [350, 420, 380, 500, 450]
empty_list = []

# 1. Get the number of courses offered
num_courses = len(course_list)
print(f"1. Number of courses: {num_courses}") # Output: 3

# 2. Get the number of student count entries
num_counts = len(student_counts)
print(f"2. Number of student count entries: {num_counts}") # Output:
5

# 3. Get the length of an empty list
empty_len = len(empty_list)
print(f"3. Length of empty list: {empty_len}") # Output: 0

# 4. Use len() in a loop condition
tasks = ["Task1", "Task2"]
while len(tasks) > 0:
    task = tasks.pop(0)
    print(f"4. Processing task: {task}, Remaining: {len(tasks)}")
# Output:
# Processing task: Task1, Remaining: 1
# Processing task: Task2, Remaining: 0

# 5. Check if list is not empty before accessing index 0
if len(course_list) > 0:
    print(f"5. First course (list not empty): {course_list[0]}")
else:
    print("5. Course list is empty.")
# Output: First course (list not empty): Data Science

# 6. Length of a list containing other lists (counts nested lists as
one item)
nested_list = [1, [2, 3], 4, [5, 6, 7]]
nested_len = len(nested_list)
print(f"6. Length of nested list: {nested_len}") # Output: 4

# 7. Length after appending an item
course_list.append("Cloud Computing") # List is now 4 items long
print(f"7. Length after append: {len(course_list)}") # Output: 4

# 8. Length after extending the list
```

```
more_courses = ["ML", "AI"]
course_list.extend(more_courses) # List is now 6 items long
print(f"8. Length after extend: {len(course_list)}") # Output: 6

# 9. Length of a list created from a range
batch_ids = list(range(101, 106)) # [101, 102, 103, 104, 105]
num_batches = len(batch_ids)
print(f"9. Number of batch IDs: {num_batches}") # Output: 5

# 10. Length of list used in conditional message
if len(student_counts) >= 5:
    print(f"10. We have data for {len(student_counts)} course
batches.")
# Output: We have data for 5 course batches.
```

i) `list.clear()`

Explanation: Removes all items from the list, making it empty. Modifies the list in-place. **Examples:**

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
tasks_to_clear = ["Task A", "Task B"]

# 1. Clear the course list
print(f"1. List before clear: {course_list}")
course_list.clear()
print(f"   List after clear: {course_list}")
# Output:
# List before clear: ['Data Science', 'Data Engineering', 'Azure
DevOps']
# List after clear: []

# 2. Clear the tasks list
print(f"2. Tasks before clear: {tasks_to_clear}")
tasks_to_clear.clear()
print(f"   Tasks after clear: {tasks_to_clear}")
# Output:
# Tasks before clear: ['Task A', 'Task B']
# Tasks after clear: []

# 3. Clear a list that is already empty
empty_list = []
print(f"3. Empty list before clear: {empty_list}")
empty_list.clear()
print(f"   Empty list after clear: {empty_list}")
# Output:
# Empty list before clear: []
# Empty list after clear: []

# 4. Use case: Resetting a list for reuse in a loop
```

```python
batch_students = []
for batch in range(2):
    # Simulate loading students for a batch
    batch_students = [f"Student_{batch}_A", f"Student_{batch}_B"]
    print(f"4. Processing batch {batch+1}: {batch_students}")
    # Clear list for next batch (if logic required it)
    batch_students.clear()
    print(f"   List after clear for batch {batch+1}:
{batch_students}")
# Output:
# Processing batch 1: ['Student_0_A', 'Student_0_B']
# List after clear for batch 1: []
# Processing batch 2: ['Student_1_A', 'Student_1_B']
# List after clear for batch 2: []


# 5. Clearing a list of numbers
numbers = [1, 2, 3, 4, 5]
numbers.clear()
print(f"5. Numbers list after clear: {numbers}") # Output: []

# 6. Clearing a list of tuples
coordinates = [(0, 0), (1, 1)]
coordinates.clear()
print(f"6. Coordinates list after clear: {coordinates}") # Output: []

# 7. Clearing a list with mixed types
mixed_list = ["Text", 10, None, True]
mixed_list.clear()
print(f"7. Mixed list after clear: {mixed_list}") # Output: []

# 8. Clearing a list referred to by another variable (affects both)
original_list = [100, 200]
ref_list = original_list # Both variables point to the same list
ref_list.clear()
print(f"8. Original list after clearing ref: {original_list}") #
Output: []
print(f"   Ref list after clearing ref: {ref_list}") # Output: []

# 9. Clearing a slice doesn't work; clear() applies to the whole list
object
data = [1, 2, 3, 4, 5]
# data[1:3].clear() # This is NOT valid syntax
data.clear() # Clears the whole list 'data'
print(f"9. Data after clearing whole list: {data}") # Output: []

# 10. Re-populating a cleared list
cleared_list = ["Old Data"]
cleared_list.clear()
cleared_list.append("New Data 1")
```

```
cleared_list.append("New Data 2")
print(f"10. Re-populated list: {cleared_list}") # Output: ['New Data 1', 'New Data 2']
```

j) Membership Testing (`in`, `not in`)

Explanation: Checks if an item exists within the list. Returns `True` or `False`.

Examples:

```
course_list = ["Data Science", "Data Engineering", "Azure DevOps"]
tech_skills = ["Python", "SQL", "Git", "CI/CD"]

# 1. Check if "Data Science" is in the course list
is_ds_offered = "Data Science" in course_list
print(f"1. Is 'Data Science' offered? {is_ds_offered}") # Output: True

# 2. Check if "Machine Learning" is in the course list
is_ml_offered = "Machine Learning" in course_list
print(f"2. Is 'Machine Learning' offered? {is_ml_offered}") # Output: False

# 3. Check if "Python" is NOT in the course list
is_python_not_in_courses = "Python" not in course_list
print(f"3. Is 'Python' not in courses? {is_python_not_in_courses}") # Output: True

# 4. Check if "Python" IS in the tech skills list
is_python_in_skills = "Python" in tech_skills
print(f"4. Is 'Python' in tech skills? {is_python_in_skills}") # Output: True

# 5. Using 'in' in an if statement
course_needed = "SQL"
if course_needed in tech_skills:
    print(f"5. '{course_needed}' is a required skill.")
else:
    print(f"5. '{course_needed}' is not listed as a required skill.") # Output: 'SQL' is a required skill.

# 6. Check for a number in a list of student counts
student_counts = [350, 420, 380]
is_420_present = 420 in student_counts
print(f"6. Is count 420 present? {is_420_present}") # Output: True

# 7. Check for None in a list
optional_features = ["Feature A", None]
is_none_present = None in optional_features
print(f"7. Is None present in features? {is_none_present}") # Output: True
```

```python
# 8. Check for a tuple in a list of tuples
coordinates = [(0, 0), (1, 1)]
point_to_check = (1, 1)
is_point_present = point_to_check in coordinates
print(f"8. Is point {point_to_check} present? {is_point_present}") #
Output: True

# 9. Check for a list within a list (checks for the exact list
object/value)
nested_list = [1, [2, 3], 4]
sub_list_check = [2, 3]
is_sublist_present = sub_list_check in nested_list
print(f"9. Is sub-list [2, 3] present? {is_sublist_present}") #
Output: True

# 10. Using 'not in' to add if missing
new_skill = "Docker"
if new_skill not in tech_skills:
    tech_skills.append(new_skill)
print(f"10. Tech skills after checking for '{new_skill}':
{tech_skills}")
# Output: ['Python', 'SQL', 'Git', 'CI/CD', 'Docker']
```

---

2. `tuple`

Explanation: Tuples are **immutable** (unchangeable), ordered sequences of items. They are defined using parentheses `()`, with items separated by commas. Like lists, they can contain items of different data types. Because they are immutable, they are often used for fixed collections of items and can be used as keys in dictionaries (whereas lists cannot).

Your example:

- `tech_skills = ("Python", "SQL", "Git", "CI/CD")`

Common Operations & Methods:

Tuples support indexing, slicing, `len()`, and membership testing (`in`, `not in`) just like lists. However, they lack methods that modify the tuple (like `append`, `remove`, `sort`, etc.). They have only two primary methods: `count()` and `index()`.

a) Creation and Indexing/Slicing

Explanation: Creating tuples and accessing elements or sub-tuples. Single-

element tuples require a trailing comma (e.g., `(item,)`). **Examples:**

```
tech_skills = ("Python", "SQL", "Git", "CI/CD", "Python")
course_info = ("DS101", "Data Science", 15, 4.8) # ID, Name, Weeks,
Rating

# 1. Create a tuple of skills
print(f"1. Tech skills tuple: {tech_skills}")

# 2. Access the second skill
second_skill = tech_skills[1]
print(f"2. Second skill: {second_skill}") # Output: SQL

# 3. Access the last skill
last_skill = tech_skills[-1]
print(f"3. Last skill: {last_skill}") # Output: Python

# 4. Get a slice of the first two skills
first_two_skills = tech_skills[:2]
print(f"4. First two skills slice: {first_two_skills}") # Output:
('Python', 'SQL')

# 5. Get course name and rating from course_info
name_rating = course_info[1], course_info[3] # Creates a new tuple
print(f"5. Course name and rating: {name_rating}") # Output: ('Data
Science', 4.8)

# 6. Create a single-element tuple (note the comma)
single_tool = ("Azure",)
print(f"6. Single element tuple: {single_tool}, Type:
{type(single_tool)}")
# Output: ('Azure',), Type: <class 'tuple'>

# 7. Create an empty tuple
empty_tuple = ()
print(f"7. Empty tuple: {empty_tuple}")

# 8. Tuple packing (creating tuple without parentheses)
student_record = "Alice", 101, "DS" # Automatically becomes a tuple
print(f"8. Tuple packing: {student_record}") # Output: ('Alice', 101,
'DS')

# 9. Tuple unpacking (assigning tuple elements to variables)
course_id, course_name, duration, rating = course_info
print(f"9. Unpacked course name: {course_name}, Duration:
{duration}")
# Output: Unpacked course name: Data Science, Duration: 15

# 10. Accessing elements in a nested tuple
nested_tuple = (1, 2, ("a", "b"), 3)
```

```
inner_element = nested_tuple[2][1] # Access 'b'
print(f"10. Element in nested tuple: {inner_element}") # Output: b
```

b) `tuple.count(item)`

Explanation: Returns the number of times the specified `item` appears in the tuple. **Examples:**

```
tech_skills = ("Python", "SQL", "Git", "CI/CD", "Python", "SQL",
"Python")
ratings = (5, 4, 5, 5, 3, 4, 5)

# 1. Count occurrences of "Python" in tech_skills
python_count = tech_skills.count("Python")
print(f"1. Count of 'Python': {python_count}") # Output: 3

# 2. Count occurrences of "SQL"
sql_count = tech_skills.count("SQL")
print(f"2. Count of 'SQL': {sql_count}") # Output: 2

# 3. Count occurrences of "Git"
git_count = tech_skills.count("Git")
print(f"3. Count of 'Git': {git_count}") # Output: 1

# 4. Count occurrences of "Java" (not present)
java_count = tech_skills.count("Java")
print(f"4. Count of 'Java': {java_count}") # Output: 0

# 5. Count occurrences of rating 5
rating_5_count = ratings.count(5)
print(f"5. Count of rating 5: {rating_5_count}") # Output: 4

# 6. Count occurrences of rating 3
rating_3_count = ratings.count(3)
print(f"6. Count of rating 3: {rating_3_count}") # Output: 1

# 7. Count occurrences of rating 1 (not present)
rating_1_count = ratings.count(1)
print(f"7. Count of rating 1: {rating_1_count}") # Output: 0

# 8. Count in a tuple with mixed types
mixed_tuple = (1, "Python", True, 1, "Python", 1)
count_1 = mixed_tuple.count(1) # Note: True is treated as 1 here
print(f"8. Count of 1 in mixed tuple: {count_1}") # Output: 4 (1,
True, 1, 1)
count_true = mixed_tuple.count(True)
print(f"   Count of True in mixed tuple: {count_true}") # Output: 4

# 9. Count in a tuple with nested tuples (counts the exact tuple)
nested = (1, (2, 3), 4, (2, 3))
```

```
nested_count = nested.count((2, 3))
print(f"9. Count of (2, 3): {nested_count}") # Output: 2

# 10. Count in an empty tuple
empty_count = ().count("anything")
print(f"10. Count in empty tuple: {empty_count}") # Output: 0
```

c) `tuple.index(item[, start[, end]])`

Explanation: Returns the index of the *first* occurrence of the specified `item`. Raises a `ValueError` if the item is not found. Optional `start` and `end` arguments limit the search to a slice. **Examples:**

```
tech_skills = ("Python", "SQL", "Git", "CI/CD", "Python", "SQL")
ratings = (5, 4, 5, 5, 3, 4, 5)

# 1. Find the index of the first occurrence of "SQL"
sql_index = tech_skills.index("SQL")
print(f"1. Index of first 'SQL': {sql_index}") # Output: 1

# 2. Find the index of the first occurrence of "Python"
python_index = tech_skills.index("Python")
print(f"2. Index of first 'Python': {python_index}") # Output: 0

# 3. Find the index of "Git"
git_index = tech_skills.index("Git")
print(f"3. Index of 'Git': {git_index}") # Output: 2

# 4. Attempt to find the index of "Java" (raises ValueError)
try:
    java_index = tech_skills.index("Java")
except ValueError as e:
    print(f"4. Error finding index of 'Java': {e}")
# Output: Error finding index of 'Java': tuple.index(x): x not in
tuple

# 5. Find the index of the first rating 5
rating_5_index = ratings.index(5)
print(f"5. Index of first rating 5: {rating_5_index}") # Output: 0

# 6. Find the index of rating 3
rating_3_index = ratings.index(3)
print(f"6. Index of rating 3: {rating_3_index}") # Output: 4

# 7. Find "Python" starting the search from index 1
python_index_from_1 = tech_skills.index("Python", 1) # Starts search
at index 1, finds it at index 4
print(f"7. Index of 'Python' starting from index 1:
{python_index_from_1}") # Output: 4
```

```python
# 8. Find "SQL" starting from index 2
sql_index_from_2 = tech_skills.index("SQL", 2) # Starts search at index 2, finds it at index 5
print(f"8. Index of 'SQL' starting from index 2: {sql_index_from_2}") # Output: 5

# 9. Find rating 4 between index 2 and 5 (exclusive end)
rating_4_slice = ratings.index(4, 2, 5) # Search in (5, 5, 3), finds nothing
try:
    rating_4_slice = ratings.index(4, 2, 5) # Search in slice (5, 5, 3)
except ValueError as e:
    print(f"9. Error finding rating 4 in slice [2:5]: {e}")
# Output: Error finding rating 4 in slice [2:5]: 4 is not in tuple

# 10. Find index of a nested tuple
nested = (1, (2, 3), 4, (2, 3))
nested_index = nested.index((2, 3))
print(f"10. Index of (2, 3): {nested_index}") # Output: 1
```

(Also supports `len()`, `in`, `not in` similar to lists)

---

3. `range`

Explanation: Represents an **immutable** sequence of numbers, typically used for looping a specific number of times in `for` loops. It's memory-efficient because it doesn't generate all the numbers at once, only when needed (it's a generator). Created using `range(stop)`, `range(start, stop)`, or `range(start, stop, step)`.

- `start`: The starting number (inclusive, defaults to 0).
- `stop`: The ending number (exclusive).
- `step`: The increment between numbers (defaults to 1).

Your example:

- `batch_ids = range(101, 106)` (Represents 101, 102, 103, 104, 105)

Common Operations & Usage:

Ranges primarily support iteration, `len()`, membership testing (`in`, `not in`), indexing, and slicing (though slicing also returns a range object).

a) Creation and Iteration

Explanation: Creating range objects and using them in `for` loops. **Examples:**

```python
# 1. Simple range (0 up to, but not including, 5)
print("1. Looping with range(5):")
for i in range(5):
    print(f"   - Iteration {i}")
# Output: Iteration 0, 1, 2, 3, 4

# 2. Range with start and stop (batch IDs 101 to 105)
batch_ids = range(101, 106)
print(f"\n2. Processing Batch IDs: {batch_ids}")
for batch_id in batch_ids:
    print(f"   - Processing Batch {batch_id}")
# Output: Processing Batch 101, 102, 103, 104, 105

# 3. Range with start, stop, and step (even numbers 0 to 8)
even_numbers = range(0, 10, 2)
print(f"\n3. Even numbers range: {even_numbers}")
for num in even_numbers:
    print(f"   - Even number: {num}")
# Output: Even number: 0, 2, 4, 6, 8

# 4. Range with negative step (countdown from 5 down to 1)
countdown = range(5, 0, -1)
print("\n4. Countdown:")
for i in countdown:
    print(f"   - T-minus {i}")
# Output: T-minus 5, 4, 3, 2, 1

# 5. Creating a range object directly
module_range = range(1, 6) # Represents modules 1 to 5
print(f"\n5. Module range object: {module_range}") # Output: range(1,
6)

# 6. Iterating through indices of a list
course_list = ["DS", "DE", "ADO"]
print("\n6. Accessing courses by index:")
for i in range(len(course_list)):
    print(f"   - Index {i}: {course_list[i]}")
# Output: Index 0: DS, Index 1: DE, Index 2: ADO

# 7. Range where start equals stop (empty sequence)
empty_range = range(5, 5)
print("\n7. Iterating over empty range:")
for i in empty_range:
    print(f"   - This won't print: {i}") # No output from loop
print(f"   (Empty range object: {empty_range})")

# 8. Range where step doesn't allow reaching stop (e.g., positive
step when start > stop)
invalid_step_range = range(10, 0, 1) # Start=10, Stop=0, Step=1 ->
Empty
```

```
print("\n8. Iterating over invalid step range:")
for i in invalid_step_range:
    print(f"   - This won't print: {i}") # No output
print(f"   (Invalid step range object: {invalid_step_range})")

# 9. Converting range to list to see all numbers
batch_list = list(range(101, 106))
print(f"\n9. Batch IDs as a list: {batch_list}") # Output: [101, 102,
103, 104, 105]

# 10. Converting range to tuple
even_tuple = tuple(range(0, 10, 2))
print(f"10. Even numbers as tuple: {even_tuple}") # Output: (0, 2, 4,
6, 8)
```

b) `len()`, Membership (`in`, `not in`), Indexing, Slicing

Explanation: Getting length, checking for number presence, accessing elements by index (without generating all numbers), and slicing range objects. **Examples:**

```
batch_ids = range(101, 106) # 101, 102, 103, 104, 105
numbers = range(0, 20, 3) # 0, 3, 6, 9, 12, 15, 18

# 1. Get the number of items in batch_ids range
num_batches = len(batch_ids)
print(f"1. Number of batch IDs: {num_batches}") # Output: 5

# 2. Get the number of items in numbers range
count_numbers = len(numbers)
print(f"2. Count of numbers in range(0, 20, 3): {count_numbers}") #
Output: 7

# 3. Check if batch ID 103 is in the range
is_103_present = 103 in batch_ids
print(f"3. Is batch ID 103 present? {is_103_present}") # Output: True

# 4. Check if batch ID 106 is in the range
is_106_present = 106 in batch_ids
print(f"4. Is batch ID 106 present? {is_106_present}") # Output:
False (range stops before 106)

# 5. Check if 10 is NOT in the 'numbers' range
is_10_not_present = 10 not in numbers
print(f"5. Is 10 not in numbers range? {is_10_not_present}") #
Output: True (only multiples of 3)

# 6. Check if 12 IS in the 'numbers' range
is_12_present = 12 in numbers
print(f"6. Is 12 in numbers range? {is_12_present}") # Output: True
```

```python
# 7. Get the first element (index 0) of batch_ids
first_batch_id = batch_ids[0]
print(f"7. First batch ID (index 0): {first_batch_id}") # Output: 101

# 8. Get the third element (index 2) of batch_ids
third_batch_id = batch_ids[2]
print(f"8. Third batch ID (index 2): {third_batch_id}") # Output: 103

# 9. Get a slice of the batch_ids range (returns another range
object)
batch_slice = batch_ids[1:4] # Represents indices 1, 2, 3 -> numbers
102, 103, 104
print(f"9. Slice of batch IDs [1:4]: {batch_slice}") # Output:
range(102, 105)
print(f"   Slice as list: {list(batch_slice)}") # Output: [102, 103,
104]

# 10. Get the last element using negative index
last_batch_id = batch_ids[-1]
print(f"10. Last batch ID (index -1): {last_batch_id}") # Output: 105
```

🎯 Set Types

Set types represent unordered collections of **unique** items. They are useful for membership testing, removing duplicates, and performing mathematical set operations (like union, intersection, difference).

1. `set`

Explanation: Sets are **mutable** (changeable), unordered collections of unique, **hashable** items. Hashable means the items must be immutable (like numbers, strings, tuples, but not lists or other sets). Sets are defined using curly braces `{}` or the `set()` constructor. An empty set *must* be created with `set()`, as `{}` creates an empty dictionary.

Your examples:

- `data_analytics_topics = {"Visualization", "Power BI", "EDA", "Reporting"}`
- `devops_tools = {"Docker", "Kubernetes", "Terraform", "GitHub Actions"}`

Common Operations & Methods:

a) Creation and `add(item)`

Explanation: Creating sets (removes duplicates automatically) and adding single elements. **Examples:**

```
# 1. Create a set of analytics topics (duplicates ignored)
data_analytics_topics = {"Visualization", "Power BI", "EDA",
"Reporting", "Power BI"}
print(f"1. Analytics topics set: {data_analytics_topics}")
# Output: {'Visualization', 'EDA', 'Reporting', 'Power BI'} (Order
may vary)

# 2. Create a set using set() from a list (removes duplicates)
course_list = ["Data Science", "Data Engineering", "Azure DevOps",
"Data Science"]
unique_courses = set(course_list)
print(f"2. Unique courses from list: {unique_courses}")
# Output: {'Data Engineering', 'Azure DevOps', 'Data Science'} (Order
may vary)

# 3. Create an empty set
required_skills = set()
print(f"3. Empty set: {required_skills}")

# 4. Add a skill to the empty set
required_skills.add("Python")
print(f"4. Set after adding 'Python': {required_skills}") # Output:
{'Python'}

# 5. Add another skill
required_skills.add("SQL")
print(f"5. Set after adding 'SQL': {required_skills}") # Output:
{'Python', 'SQL'} (Order may vary)

# 6. Add an existing skill (set remains unchanged)
required_skills.add("Python")
print(f"6. Set after adding 'Python' again: {required_skills}") #
Output: {'Python', 'SQL'}

# 7. Add a number to a set
batch_ids = {101, 102}
batch_ids.add(103)
print(f"7. Batch IDs after add: {batch_ids}") # Output: {101, 102,
103}

# 8. Add a tuple (hashable) to a set
config_params = {("host", "localhost")}
config_params.add(("port", 8080))
print(f"8. Config params set: {config_params}")
```

```python
# Output: {('host', 'localhost'), ('port', 8080)}

# 9. Create set from a string (unique characters)
unique_chars = set("Edukron Institute")
print(f"9. Unique chars in 'Edukron Institute': {unique_chars}")
# Output: {'r', 's', 'o', 't', 'n', 'k', 'I', 'u', 'd', ' ', 'E',
'i'} (Order may vary)

# 10. Attempt to add unhashable type (list) -> TypeError
try:
    required_skills.add(["Git", "CI/CD"])
except TypeError as e:
    print(f"10. Error adding list to set: {e}")
# Output: Error adding list to set: unhashable type: 'list'
```

b) `update(iterable)`

Explanation: Adds all unique items from an `iterable` (like a list, tuple, set, or string) into the set. Modifies the set in-place. Equivalent to `set1 |= set2 | set3 ...` **Examples:**

```python
devops_tools = {"Docker", "Kubernetes"}
data_tools = {"Python", "SQL", "Pandas"}
new_tools_list = ["Terraform", "GitHub Actions", "Docker"] # Note
duplicate "Docker"
more_skills = ("Git", "SQL") # Note duplicate "SQL"

# 1. Update devops_tools with new_tools_list
devops_tools.update(new_tools_list)
print(f"1. DevOps tools after update: {devops_tools}")
# Output: {'GitHub Actions', 'Docker', 'Terraform', 'Kubernetes'}
(Order may vary)

# 2. Update data_tools with more_skills tuple
data_tools.update(more_skills)
print(f"2. Data tools after update: {data_tools}")
# Output: {'Pandas', 'Git', 'SQL', 'Python'} (Order may vary)

# 3. Update with another set
all_tools = data_tools.copy() # Start with data tools
all_tools.update(devops_tools)
print(f"3. All tools combined using update: {all_tools}")
# Output: {'Pandas', 'Git', 'SQL', 'Python', 'GitHub Actions',
'Docker', 'Terraform', 'Kubernetes'}

# 4. Update with characters from a string
chars = {'a', 'b'}
chars.update("ace") # Adds 'c', 'e' ('a' already exists)
print(f"4. Chars updated with string 'ace': {chars}") # Output: {'b',
'c', 'a', 'e'}
```

```python
# 5. Update with an empty iterable (no change)
initial_set = {1, 2}
initial_set.update([])
print(f"5. Set updated with empty list: {initial_set}") # Output: {1,
2}

# 6. Update an empty set
empty_set = set()
empty_set.update([101, 102, 101])
print(f"6. Updating an empty set: {empty_set}") # Output: {101, 102}

# 7. Update with items from a range
numbers = {1, 5}
numbers.update(range(3, 6)) # Adds 3, 4 (5 already exists)
print(f"7. Numbers updated with range(3, 6): {numbers}") # Output:
{1, 3, 4, 5}

# 8. Update with multiple iterables at once
features = {"Login"}
features.update(["Profile", "Settings"], ("Logout", "Profile"))
print(f"8. Features updated with multiple iterables: {features}")
# Output: {'Logout', 'Settings', 'Profile', 'Login'}

# 9. Update using the |= operator (equivalent)
set_a = {1, 2}
set_b = {2, 3}
set_a |= set_b # set_a = set_a.union(set_b)
print(f"9. Set A updated using |= : {set_a}") # Output: {1, 2, 3}

# 10. Attempt to update with non-iterable (TypeError)
try:
    tools = {"Git"}
    tools.update(123)
except TypeError as e:
    print(f"10. Error updating with non-iterable: {e}")
# Output: Error updating with non-iterable: 'int' object is not
iterable
```

c) `remove(item)` vs `discard(item)`

Explanation: Both remove an item from the set.

- `remove(item)`: Removes `item`. Raises a `KeyError` if `item` is not found.
- `discard(item)`: Removes `item` if present. Does *not* raise an error if `item` is not found. Both modify the set in-place. **Examples:**

```python
devops_tools = {"Docker", "Kubernetes", "Terraform", "GitHub
Actions"}
```

```python
# 1. Remove "Terraform" using remove()
devops_tools.remove("Terraform")
print(f"1. Set after removing 'Terraform': {devops_tools}")
# Output: {'GitHub Actions', 'Kubernetes', 'Docker'} (Order may vary)

# 2. Attempt to remove "Terraform" again using remove() (KeyError)
try:
    devops_tools.remove("Terraform")
except KeyError as e:
    print(f"2. Error removing 'Terraform' again: {e}")
# Output: Error removing 'Terraform' again: 'Terraform'

# 3. Discard "Kubernetes" using discard()
devops_tools.discard("Kubernetes")
print(f"3. Set after discarding 'Kubernetes': {devops_tools}")
# Output: {'GitHub Actions', 'Docker'}

# 4. Attempt to discard "Kubernetes" again (no error)
devops_tools.discard("Kubernetes")
print(f"4. Set after discarding 'Kubernetes' again: {devops_tools}")
# No change, no error
# Output: {'GitHub Actions', 'Docker'}

# 5. Discard an item not present ("Ansible")
devops_tools.discard("Ansible")
print(f"5. Set after discarding 'Ansible' (not present):
{devops_tools}") # No change, no error
# Output: {'GitHub Actions', 'Docker'}

# 6. Remove "Docker" using remove()
devops_tools.remove("Docker")
print(f"6. Set after removing 'Docker': {devops_tools}")
# Output: {'GitHub Actions'}

# 7. Remove the last item "GitHub Actions"
devops_tools.remove("GitHub Actions")
print(f"7. Set after removing last item: {devops_tools}") # Output:
set()

# 8. Discard from an empty set (no error)
devops_tools.discard("Anything")
print(f"8. Discarding from empty set: {devops_tools}") # Output:
set()

# 9. Remove from an empty set (KeyError)
try:
    devops_tools.remove("Anything")
except KeyError as e:
    print(f"9. Error removing from empty set: {e}")
```

```
# Output: Error removing from empty set: 'Anything'

# 10. Conditional removal using discard
topics = {"EDA", "Visualization", "Power BI"}
topic_to_remove = "SQL" # Might not be present
topics.discard(topic_to_remove) # Safely removes if present, does
nothing otherwise
print(f"10. Topics after safely discarding '{topic_to_remove}':
{topics}")
# Output: {'Power BI', 'EDA', 'Visualization'}
```

d) `pop()`

Explanation: Removes and *returns* an **arbitrary** element from the set. Raises a `KeyError` if the set is empty. Since sets are unordered, you cannot predict which element will be popped. Modifies the set in-place. **Examples:**

```
data_analytics_topics = {"Visualization", "Power BI", "EDA",
"Reporting"}
numbers = {101, 102, 103}

# 1. Pop an arbitrary topic
popped_topic = data_analytics_topics.pop()
print(f"1. Popped topic: '{popped_topic}', Remaining:
{data_analytics_topics}")
# Output might be: Popped topic: 'Visualization', Remaining: {'EDA',
'Reporting', 'Power BI'} (Order/Item vary)

# 2. Pop another arbitrary topic
popped_topic_2 = data_analytics_topics.pop()
print(f"2. Popped another: '{popped_topic_2}', Remaining:
{data_analytics_topics}")
# Output might be: Popped another: 'EDA', Remaining: {'Reporting',
'Power BI'}

# 3. Pop an arbitrary number
popped_number = numbers.pop()
print(f"3. Popped number: {popped_number}, Remaining: {numbers}") #
e.g., 101, {102, 103}

# 4. Pop another number
popped_number_2 = numbers.pop()
print(f"4. Popped another number: {popped_number_2}, Remaining:
{numbers}") # e.g., 102, {103}

# 5. Pop the last remaining number
last_number = numbers.pop()
print(f"5. Popped last number: {last_number}, Remaining: {numbers}")
# e.g., 103, set()
```

```python
# 6. Attempt to pop from the now empty set (KeyError)
try:
    numbers.pop()
except KeyError as e:
    print(f"6. Error popping from empty set: {e}")
# Output: Error popping from empty set: 'pop from an empty set'

# 7. Pop from a set with one element
single_set = {"Edukron"}
item = single_set.pop()
print(f"7. Popped from single set: '{item}', Remaining:
{single_set}")
# Output: Popped from single set: 'Edukron', Remaining: set()

# 8. Use pop in a loop to process all items (order not guaranteed)
topics_to_process = {"Git", "SQL", "Python"}
print("8. Processing topics using pop:")
while topics_to_process: # While set is not empty
    topic = topics_to_process.pop()
    print(f"   - Processing: {topic}")
# Output: Order will vary, e.g.: Processing: Git, Processing: SQL,
Processing: Python

# 9. Pop from a set containing tuples
coord_set = {(0,0), (1,1), (2,2)}
popped_coord = coord_set.pop()
print(f"9. Popped coordinate: {popped_coord}, Remaining:
{coord_set}")

# 10. Pop from a set containing mixed types
mixed_set = {1, "hello", None, True} # True might collide with 1
popped_item = mixed_set.pop()
print(f"10. Popped from mixed set: {popped_item}, Type:
{type(popped_item)}, Remaining: {mixed_set}")
```

e) Set Operations (Union, Intersection, Difference, Symmetric Difference)

Explanation: Mathematical operations on sets.

- **Union (`|` or `union()`):** Returns a new set with all items from both sets.
- **Intersection (`&` or `intersection()`):** Returns a new set with items common to both sets.
- **Difference (`-` or `difference()`):** Returns a new set with items in the first set but *not* in the second.
- **Symmetric Difference (`^` or `symmetric_difference()`):** Returns a new set with items in either set, but *not* in both. In-place versions exist

for some (e.g., `update()` is like union, `intersection_update()`, `difference_update()`, `symmetric_difference_update()`).

Examples:

```
data_sci_skills = {"Python", "Pandas", "ML", "SQL"}
data_eng_skills = {"Python", "SQL", "ADF", "ADLS", "Databricks"}
devops_skills = {"GitHub", "Pipelines", "Terraform", "Python"}

# 1. Union of DS and DE skills (all unique skills)
all_data_skills = data_sci_skills.union(data_eng_skills)
# or: all_data_skills = data_sci_skills | data_eng_skills
print(f"1. Union (DS | DE): {all_data_skills}")
# Output: {'ML', 'ADLS', 'Python', 'ADF', 'Pandas', 'Databricks',
'SQL'} (Order varies)

# 2. Intersection of DS and DE skills (common skills)
common_data_skills = data_sci_skills.intersection(data_eng_skills)
# or: common_data_skills = data_sci_skills & data_eng_skills
print(f"2. Intersection (DS & DE): {common_data_skills}")
# Output: {'Python', 'SQL'}

# 3. Skills unique to Data Science (DS - DE)
ds_only_skills = data_sci_skills.difference(data_eng_skills)
# or: ds_only_skills = data_sci_skills - data_eng_skills
print(f"3. Difference (DS - DE): {ds_only_skills}")
# Output: {'Pandas', 'ML'}

# 4. Skills unique to Data Engineering (DE - DS)
de_only_skills = data_eng_skills.difference(data_sci_skills)
# or: de_only_skills = data_eng_skills - data_sci_skills
print(f"4. Difference (DE - DS): {de_only_skills}")
# Output: {'ADF', 'Databricks', 'ADLS'}

# 5. Symmetric Difference (skills in DS or DE, but not both)
non_common_data_skills =
data_sci_skills.symmetric_difference(data_eng_skills)
# or: non_common_data_skills = data_sci_skills ^ data_eng_skills
print(f"5. Symmetric Difference (DS ^ DE): {non_common_data_skills}")
# Output: {'ML', 'ADLS', 'ADF', 'Pandas', 'Databricks'}

# 6. Intersection of all three skill sets
common_all_three = data_sci_skills & data_eng_skills & devops_skills
print(f"6. Intersection (DS & DE & DevOps): {common_all_three}")
# Output: {'Python'}

# 7. Skills in DS but not in DevOps (DS - DevOps)
ds_not_devops = data_sci_skills - devops_skills
print(f"7. Difference (DS - DevOps): {ds_not_devops}")
# Output: {'Pandas', 'ML', 'SQL'}
```

```python
# 8. Update DS skills in-place with DE skills (DS becomes the union)
ds_skills_copy = data_sci_skills.copy()
ds_skills_copy.update(data_eng_skills) # Or ds_skills_copy |= data_eng_skills
print(f"8. DS skills updated with DE skills: {ds_skills_copy}")
# Output: {'ML', 'ADLS', 'Python', 'ADF', 'Pandas', 'Databricks', 'SQL'}

# 9. Keep only common skills in DS (in-place intersection)
ds_skills_copy = data_sci_skills.copy()
ds_skills_copy.intersection_update(data_eng_skills) # Or ds_skills_copy &= data_eng_skills
print(f"9. DS skills updated to intersection with DE: {ds_skills_copy}")
# Output: {'SQL', 'Python'}

# 10. Remove skills found in DevOps from DS (in-place difference)
ds_skills_copy = data_sci_skills.copy()
ds_skills_copy.difference_update(devops_skills) # Or ds_skills_copy -= devops_skills
print(f"10. DS skills after removing DevOps skills: {ds_skills_copy}")
# Output: {'ML', 'Pandas', 'SQL'}
```

(Also supports `len()`, `in`, `not in`)

---

2. `frozenset`

Explanation: Frozensets are **immutable** (unchangeable), unordered collections of unique, hashable items. They are created using the `frozenset()` constructor. Because they are immutable and hashable, they can be used as elements within other sets or as keys in dictionaries.

Your example:

- `platforms = frozenset(["Azure", "Python", "Databricks"])`

Common Operations & Usage:

Frozensets support non-modifying set operations (union, intersection, difference, etc. - returning new frozensets or sets), `len()`, and membership testing (`in`, `not in`). They do *not* support methods like `add`, `remove`, `update`, `pop`, etc.

a) Creation and Immutability

Explanation: Creating frozensets and demonstrating their immutability.

Examples:

```
# 1. Create a frozenset from a list
platforms = frozenset(["Azure", "Python", "Databricks", "Azure"]) #
Duplicate ignored
print(f"1. Platforms frozenset: {platforms}")
# Output: frozenset({'Python', 'Databricks', 'Azure'}) (Order may
vary)

# 2. Create a frozenset from a tuple
core_skills = frozenset(("SQL", "Python"))
print(f"2. Core skills frozenset: {core_skills}")
# Output: frozenset({'SQL', 'Python'})

# 3. Create from a string (unique chars)
fs_chars = frozenset("Edukron")
print(f"3. Frozenset from 'Edukron': {fs_chars}")
# Output: frozenset({'k', 'n', 'd', 'E', 'r', 'o', 'u'})

# 4. Create an empty frozenset
empty_fs = frozenset()
print(f"4. Empty frozenset: {empty_fs}") # Output: frozenset()

# 5. Attempt to add an element (AttributeError)
try:
    platforms.add("AWS")
except AttributeError as e:
    print(f"5. Error adding to frozenset: {e}")
# Output: Error adding to frozenset: 'frozenset' object has no
attribute 'add'

# 6. Attempt to remove an element (AttributeError)
try:
    platforms.remove("Azure")
except AttributeError as e:
    print(f"6. Error removing from frozenset: {e}")
# Output: Error removing from frozenset: 'frozenset' object has no
attribute 'remove'

# 7. Attempt to update (AttributeError)
try:
    platforms.update(["AWS"])
except AttributeError as e:
    print(f"7. Error updating frozenset: {e}")
# Output: Error updating frozenset: 'frozenset' object has no
attribute 'update'

# 8. Create a set containing frozensets (possible because frozensets
are hashable)
skill_combinations = {core_skills, frozenset(["ML", "Python"]),
```

```
frozenset(["SQL"])}
print(f"8. Set of frozensets: {skill_combinations}")
# Output: {frozenset({'SQL'}), frozenset({'Python', 'SQL'}),
frozenset({'ML', 'Python'})}

# 9. Use frozenset as dictionary key
course_tools = {
    frozenset(["Python", "Pandas"]): "Data Analysis basics",
    platforms: "Cloud Data Engineering platform"
}
print(f"9. Dict with frozenset keys: {course_tools}")
print(f"   Value for platforms key: {course_tools[platforms]}")
# Output: Cloud Data Engineering platform

# 10. Frozenset from range
fs_range = frozenset(range(3)) # 0, 1, 2
print(f"10. Frozenset from range(3): {fs_range}") # Output:
frozenset({0, 1, 2})
```

b) Set Operations with Frozensets

Explanation: Performing set operations; the result type depends on the operand types (frozenset op frozenset -> frozenset, frozenset op set -> set). **Examples:**

```
platforms = frozenset(["Azure", "Python", "Databricks"])
cloud_tools = frozenset(["Azure", "AWS", "GCP"])
scripting = {"Python", "Bash"} # A regular set

# 1. Union of two frozensets (returns frozenset)
all_platforms = platforms.union(cloud_tools)
# or: all_platforms = platforms | cloud_tools
print(f"1. Union (fs | fs): {all_platforms}, Type:
{type(all_platforms)}")
# Output: frozenset({'AWS', 'GCP', 'Azure', 'Python', 'Databricks'}),
Type: <class 'frozenset'>

# 2. Intersection of two frozensets (returns frozenset)
common_platforms = platforms.intersection(cloud_tools)
# or: common_platforms = platforms & cloud_tools
print(f"2. Intersection (fs & fs): {common_platforms}, Type:
{type(common_platforms)}")
# Output: frozenset({'Azure'}), Type: <class 'frozenset'>

# 3. Difference between two frozensets (returns frozenset)
platforms_only = platforms.difference(cloud_tools)
# or: platforms_only = platforms - cloud_tools
print(f"3. Difference (platforms - cloud_tools): {platforms_only},
Type: {type(platforms_only)}")
# Output: frozenset({'Python', 'Databricks'}), Type: <class
'frozenset'>
```

```
# 4. Symmetric difference between two frozensets (returns frozenset)
non_common_platforms = platforms.symmetric_difference(cloud_tools)
# or: non_common_platforms = platforms ^ cloud_tools
print(f"4. Symmetric Difference (fs ^ fs): {non_common_platforms},
Type: {type(non_common_platforms)}")
# Output: frozenset({'AWS', 'GCP', 'Python', 'Databricks'}), Type:
<class 'frozenset'>

# 5. Union of frozenset and set (returns set)
platform_scripting = platforms.union(scripting)
# or: platform_scripting = platforms | scripting
print(f"5. Union (fs | set): {platform_scripting}, Type:
{type(platform_scripting)}")
# Output: {'Databricks', 'Azure', 'Bash', 'Python'}, Type: <class
'set'>

# 6. Intersection of frozenset and set (returns set)
common_platform_scripting = platforms.intersection(scripting)
# or: common_platform_scripting = platforms & scripting
print(f"6. Intersection (fs & set): {common_platform_scripting},
Type: {type(common_platform_scripting)}")
# Output: {'Python'}, Type: <class 'set'>

# 7. Difference (frozenset - set) (returns set)
platforms_not_scripting = platforms.difference(scripting)
# or: platforms_not_scripting = platforms - scripting
print(f"7. Difference (fs - set): {platforms_not_scripting}, Type:
{type(platforms_not_scripting)}")
# Output: {'Azure', 'Databricks'}, Type: <class 'set'>

# 8. Symmetric Difference (frozenset ^ set) (returns set)
non_common_platform_scripting =
platforms.symmetric_difference(scripting)
# or: non_common_platform_scripting = platforms ^ scripting
print(f"8. Symmetric Difference (fs ^ set):
{non_common_platform_scripting}, Type:
{type(non_common_platform_scripting)}")
# Output: {'Databricks', 'Azure', 'Bash'}, Type: <class 'set'>

# 9. Checking subset relationship
is_subset = frozenset(["Python", "Azure"]).issubset(platforms)
print(f"9. Is {{'Python', 'Azure'}} subset of platforms?
{is_subset}") # Output: True

# 10. Checking superset relationship
is_superset = platforms.issuperset({"Python"})
print(f"10. Is platforms superset of {{'Python'}}? {is_superset}") #
Output: True
```

(Also supports `len()`, `in`, `not in`)

🗺️ Mapping Type (`dict`)

Explanation: Dictionaries (`dict`) store collections of **key-value pairs**. Keys must be unique and **hashable** (immutable types like strings, numbers, tuples containing only hashables). Values can be of any type and are accessed using their corresponding key. Dictionaries are **mutable**. As of Python 3.7+, dictionaries remember the insertion order of items.

Your example:

```
learning_paths = {
    "Data Science": ["Python", "Pandas", "ML"],
    "Data Engineering": ["ADF", "ADLS", "Databricks"],
    "Azure DevOps": ["GitHub", "Pipelines", "Terraform"]
}
```

Common Operations & Methods:

a) Creation, Accessing (`[]`, `get()`), Adding/Updating

Explanation: Creating dictionaries, retrieving values using keys, and adding new key-value pairs or updating existing ones. `[]` raises `KeyError` if key is missing, `get(key, default=None)` returns `None` (or a specified default) instead.

Examples:

```
learning_paths = {
    "Data Science": ["Python", "Pandas", "ML"],
    "Data Engineering": ["ADF", "ADLS", "Databricks"],
}
student_ratings = {"Alice": 4.5, "Bob": 4.8}

# 1. Create a dictionary of learning paths
print(f"1. Learning paths: {learning_paths}")

# 2. Access the tools for Data Science using []
ds_tools = learning_paths["Data Science"]
print(f"2. Data Science tools: {ds_tools}") # Output: ['Python',
'Pandas', 'ML']

# 3. Access a non-existent key using [] (KeyError)
try:
    cloud_tools = learning_paths["Cloud Computing"]
except KeyError as e:
    print(f"3. Error accessing 'Cloud Computing' with []: {e}")
# Output: Error accessing 'Cloud Computing' with []: 'Cloud
```

```
Computing'

# 4. Access the tools for Data Engineering using get()
de_tools = learning_paths.get("Data Engineering")
print(f"4. Data Engineering tools (get): {de_tools}") # Output:
['ADF', 'ADLS', 'Databricks']

# 5. Access non-existent key using get() (returns None)
cloud_tools_get = learning_paths.get("Cloud Computing")
print(f"5. Cloud Computing tools (get): {cloud_tools_get}") # Output:
None

# 6. Access non-existent key using get() with a default value
ml_tools_get = learning_paths.get("Machine Learning", []) # Default
to empty list
print(f"6. Machine Learning tools (get with default):
{ml_tools_get}") # Output: []

# 7. Add a new learning path (Azure DevOps)
learning_paths["Azure DevOps"] = ["GitHub", "Pipelines", "Terraform"]
print(f"7. Learning paths after adding ADO: {learning_paths}")

# 8. Update the tools for Data Science
learning_paths["Data Science"] = ["Python", "Pandas", "ML",
"Visualization"]
print(f"8. Learning paths after updating DS: {learning_paths}")

# 9. Add a new student rating
student_ratings["Charlie"] = 4.2
print(f"9. Student ratings after adding Charlie: {student_ratings}")
# Output: {'Alice': 4.5, 'Bob': 4.8, 'Charlie': 4.2}

# 10. Update Bob's rating
student_ratings["Bob"] = 4.9
print(f"10. Student ratings after updating Bob: {student_ratings}")
# Output: {'Alice': 4.5, 'Bob': 4.9, 'Charlie': 4.2}
```

b) `pop(key[, default])`

Explanation: Removes the item with the specified `key` and returns its value. If the key is not found, it raises a `KeyError`, unless a `default` value is provided, in which case the default value is returned. Modifies the dictionary in-place.

Examples:

```
learning_paths = {
    "Data Science": ["Python", "Pandas", "ML"],
    "Data Engineering": ["ADF", "ADLS", "Databricks"],
    "Azure DevOps": ["GitHub", "Pipelines", "Terraform"]
}
```

```python
# 1. Pop the "Data Engineering" path and get its tools
de_tools = learning_paths.pop("Data Engineering")
print(f"1. Popped DE tools: {de_tools}")
print(f"   Dict after pop: {learning_paths}")
# Output:
# Popped DE tools: ['ADF', 'ADLS', 'Databricks']
# Dict after pop: {'Data Science': ['Python', 'Pandas', 'ML'], 'Azure
DevOps': ['GitHub', 'Pipelines', 'Terraform']}

# 2. Pop "Data Science"
ds_tools = learning_paths.pop("Data Science")
print(f"2. Popped DS tools: {ds_tools}")
print(f"   Dict after pop: {learning_paths}")
# Output:
# Popped DS tools: ['Python', 'Pandas', 'ML']
# Dict after pop: {'Azure DevOps': ['GitHub', 'Pipelines',
'Terraform']}

# 3. Attempt to pop "Data Science" again (KeyError)
try:
    learning_paths.pop("Data Science")
except KeyError as e:
    print(f"3. Error popping 'Data Science' again: {e}")
# Output: Error popping 'Data Science' again: 'Data Science'

# 4. Pop a non-existent key ("Cloud") with a default value
cloud_tools = learning_paths.pop("Cloud Computing", "Path not found")
print(f"4. Popping 'Cloud Computing' with default: {cloud_tools}")
print(f"   Dict state unchanged: {learning_paths}")
# Output:
# Popping 'Cloud Computing' with default: Path not found
# Dict state unchanged: {'Azure DevOps': ['GitHub', 'Pipelines',
'Terraform']}

# 5. Pop the last remaining path ("Azure DevOps")
ado_tools = learning_paths.pop("Azure DevOps")
print(f"5. Popped ADO tools: {ado_tools}")
print(f"   Dict after pop: {learning_paths}")
# Output:
# Popped ADO tools: ['GitHub', 'Pipelines', 'Terraform']
# Dict after pop: {}

# 6. Attempt to pop from an empty dictionary (KeyError)
try:
    learning_paths.pop("Anything")
except KeyError as e:
    print(f"6. Error popping from empty dict: {e}")
# Output: Error popping from empty dict: 'Anything'

# 7. Pop from empty dict with default value
```

```
result = learning_paths.pop("Anything", None)
print(f"7. Popping from empty dict with default: {result}") # Output:
None

# 8. Pop based on a variable key
path_to_remove = "Data Engineering" # Assume dict was reset
learning_paths = {"Data Science": [], "Data Engineering": []}
if path_to_remove in learning_paths:
    removed_value = learning_paths.pop(path_to_remove)
    print(f"8. Removed value for '{path_to_remove}':
{removed_value}")
else:
    print(f"8. Path '{path_to_remove}' not found.")
# Output: Removed value for 'Data Engineering': []

# 9. Pop an item and immediately use its value
student_ratings = {"Alice": 4.5, "Bob": 4.8}
bob_rating = student_ratings.pop("Bob")
print(f"9. Bob's rating was {bob_rating}, remaining:
{student_ratings}")
# Output: Bob's rating was 4.8, remaining: {'Alice': 4.5}

# 10. Using pop to iterate and remove (less common than iterating
over keys/items)
ratings_copy = {"Alice": 4.5, "Bob": 4.8}.copy()
keys_to_pop = list(ratings_copy.keys()) # Need to copy keys first
print("10. Popping based on key list:")
for key in keys_to_pop:
    value = ratings_copy.pop(key)
    print(f"    Popped {key}: {value}")
print(f"    Dict after loop: {ratings_copy}")
# Output:
# Popping based on key list:
#     Popped Alice: 4.5
#     Popped Bob: 4.8
#     Dict after loop: {}
```

c) `popitem()`

Explanation: Removes and returns an arbitrary (key, value) pair as a tuple. In Python 3.7+, it follows LIFO (Last-In, First-Out) order - removing the most recently added item. Raises a `KeyError` if the dictionary is empty. Modifies the dictionary in-place. **Examples:**

```
learning_paths = {
    "Data Science": ["Python", "Pandas", "ML"],
    "Data Engineering": ["ADF", "ADLS", "Databricks"],
    "Azure DevOps": ["GitHub", "Pipelines", "Terraform"] # Last added
}
```

```python
# 1. Pop the last inserted item (LIFO in Python 3.7+)
key, value = learning_paths.popitem()
print(f"1. Popped item: ({key}: {value})")
print(f"   Dict after popitem: {learning_paths}")
# Output (Python 3.7+):
# Popped item: (Azure DevOps: ['GitHub', 'Pipelines', 'Terraform'])
# Dict after popitem: {'Data Science': ['Python', 'Pandas', 'ML'],
'Data Engineering': ['ADF', 'ADLS', 'Databricks']}

# 2. Pop the next last inserted item
key, value = learning_paths.popitem()
print(f"2. Popped item: ({key}: {value})")
print(f"   Dict after popitem: {learning_paths}")
# Output (Python 3.7+):
# Popped item: (Data Engineering: ['ADF', 'ADLS', 'Databricks'])
# Dict after popitem: {'Data Science': ['Python', 'Pandas', 'ML']}

# 3. Pop the remaining item
key, value = learning_paths.popitem()
print(f"3. Popped item: ({key}: {value})")
print(f"   Dict after popitem: {learning_paths}")
# Output (Python 3.7+):
# Popped item: (Data Science: ['Python', 'Pandas', 'ML'])
# Dict after popitem: {}

# 4. Attempt to popitem from an empty dictionary (KeyError)
try:
    learning_paths.popitem()
except KeyError as e:
    print(f"4. Error popping item from empty dict: {e}")
# Output: Error popping item from empty dict: 'popitem(): dictionary
is empty'

# 5. Use popitem in a loop to process all items (LIFO order in 3.7+)
paths_copy = { "DS": [], "DE": [], "ADO": [] }.copy()
print("5. Processing paths using popitem (LIFO):")
while paths_copy: # While dict is not empty
    path, tools = paths_copy.popitem()
    print(f"   - Processing: {path}")
# Output (Python 3.7+):
# Processing paths using popitem (LIFO):
#    - Processing: ADO
#    - Processing: DE
#    - Processing: DS

# 6. Popitem from a dict with integer keys
num_dict = {1: 'a', 2: 'b', 3: 'c'}
k, v = num_dict.popitem()
print(f"6. Popped item from num_dict: ({k}: {v}), Remaining:
{num_dict}")
```

```
# Output: Popped item from num_dict: (3: c), Remaining: {1: 'a', 2:
'b'}

# 7. Popitem from a dict with tuple keys
tuple_key_dict = {(0,0): "Origin", (1,1): "Point A"}
k, v = tuple_key_dict.popitem()
print(f"7. Popped item from tuple_key_dict: ({k}: {v}), Remaining:
{tuple_key_dict}")
# Output: Popped item from tuple_key_dict: ((1, 1): Point A),
Remaining: {(0, 0): 'Origin'}

# 8. Dict created in different order
paths_alt_order = {}
paths_alt_order["DS"] = []
paths_alt_order["DE"] = []
paths_alt_order["ADO"] = [] # ADO added last
k, v = paths_alt_order.popitem()
print(f"8. Popped from alt order dict: ({k}: {v})") # Output: (ADO:
[])

# 9. Single item dictionary popitem
single_item_dict = {"Edukron": "Institute"}
k, v = single_item_dict.popitem()
print(f"9. Popped from single item dict: ({k}: {v}), Remaining:
{single_item_dict}")
# Output: Popped from single item dict: (Edukron: Institute),
Remaining: {}

# 10. Storing popped key/value separately
d = {'x': 1, 'y': 2}
popped_pair = d.popitem()
popped_key = popped_pair[0]
popped_value = popped_pair[1]
print(f"10. Popped Key: {popped_key}, Popped Value: {popped_value}")
# Output: Popped Key: y, Popped Value: 2
```

d) `keys()`, `values()`, `items()`

Explanation: These methods return "view objects" that provide dynamic views of the dictionary's keys, values, or key-value (item) pairs respectively. Views reflect changes made to the dictionary. They can be iterated over or converted to lists/tuples. **Examples:**

```
learning_paths = {
    "Data Science": ["Python", "Pandas", "ML"],
    "Data Engineering": ["ADF", "ADLS", "Databricks"],
    "Azure DevOps": ["GitHub", "Pipelines", "Terraform"]
}

# 1. Get a view of the keys
```

```python
path_keys = learning_paths.keys()
print(f"1. Keys view: {path_keys}") # Output: dict_keys(['Data
Science', 'Data Engineering', 'Azure DevOps'])
# Convert to list
path_key_list = list(path_keys)
print(f"   Keys as list: {path_key_list}")

# 2. Get a view of the values
path_values = learning_paths.values()
print(f"\n2. Values view: {path_values}") # Output:
dict_values([['Python', 'Pandas', 'ML'], [...], [...]])
# Convert to list
path_value_list = list(path_values)
print(f"   Values as list: {path_value_list}")

# 3. Get a view of the key-value items (tuples)
path_items = learning_paths.items()
print(f"\n3. Items view: {path_items}") # Output: dict_items([('Data
Science', [...]), ('Data Engineering', [...]), ...])
# Convert to list
path_item_list = list(path_items)
print(f"   Items as list: {path_item_list}")

# 4. Iterate directly over keys view
print("\n4. Iterating over keys:")
for path_name in learning_paths.keys():
    print(f"   - Path: {path_name}")

# 5. Iterate directly over values view
print("\n5. Iterating over values:")
for tool_list in learning_paths.values():
    print(f"   - Tools: {tool_list}")

# 6. Iterate directly over items view (unpacking key/value)
print("\n6. Iterating over items:")
for path_name, tool_list in learning_paths.items():
    print(f"   - Path: {path_name}, First tool: {tool_list[0] if
tool_list else 'N/A'}")

# 7. Views reflect dictionary changes
student_ratings = {"Alice": 4.5, "Bob": 4.8}
ratings_view = student_ratings.values()
print(f"\n7. Initial ratings view: {ratings_view}") # Output:
dict_values([4.5, 4.8])
student_ratings["Charlie"] = 4.2 # Modify the dictionary
print(f"   Ratings view after adding Charlie: {ratings_view}") #
Output: dict_values([4.5, 4.8, 4.2])

# 8. Check membership in keys view (efficient)
has_ds = "Data Science" in learning_paths.keys() # More explicit than
```

```
`in learning_paths`
print(f"\n8. Is 'Data Science' in keys view? {has_ds}") # Output:
True

# 9. Check membership in values view (less common/efficient)
# This checks if the exact list object/value exists as a value
is_list_present = ["Python", "Pandas", "ML"] in
learning_paths.values()
print(f"9. Is ['Python', 'Pandas', 'ML'] in values view?
{is_list_present}") # Output: True

# 10. Using items to create a list of formatted strings
description_list = [f"{k}: {v[0]}..." for k, v in
learning_paths.items()]
print(f"\n10. List from items view: {description_list}")
# Output: ['Data Science: Python...', 'Data Engineering: ADF...',
'Azure DevOps: GitHub...']
```

(Also supports `len()`, `in` (checks keys), `not in`)

✅ Boolean Type (`bool`)

Explanation: Represents one of two truth values: `True` or `False`. Booleans are fundamental for conditional logic (`if`, `while`), comparisons, and representing status flags. They are a subclass of integers where `True` corresponds to `1` and `False` corresponds to `0`.

Your examples:

- `is_data_science_active = True`
- `is_azure_devops_course_full = False`
- `is_python_essential = True`

Common Operations & Usage:

a) Literals and Comparisons

Explanation: Using the literal values `True` and `False`, and generating booleans through comparison operators (`==`, `!=`, `<`, `<=`, `>`, `>=`). **Examples:**

```
total_students = 1500
capacity = 1500
average_rating = 4.8
min_rating = 4.5

# 1. Boolean literal True
is_data_science_active = True
```

```
print(f"1. DS Active Status: {is_data_science_active}") # Output:
True

# 2. Boolean literal False
is_azure_devops_course_full = False
print(f"2. ADO Full Status: {is_azure_devops_course_full}") # Output:
False

# 3. Equality check (==)
at_capacity = (total_students == capacity)
print(f"3. Is course at capacity? {at_capacity}") # Output: True

# 4. Inequality check (!=)
has_vacancies = (total_students != capacity)
print(f"4. Are there vacancies? {has_vacancies}") # Output: False

# 5. Greater than check (>)
rating_is_good = (average_rating > min_rating)
print(f"5. Is average rating good? {rating_is_good}") # Output: True

# 6. Less than check (<)
needs_improvement = (average_rating < 4.0)
print(f"6. Does rating need improvement? {needs_improvement}") #
Output: False

# 7. Greater than or equal check (>=)
meets_min_rating = (average_rating >= min_rating)
print(f"7. Does rating meet minimum? {meets_min_rating}") # Output:
True

# 8. Less than or equal check (<=)
enrollment_below_target = (total_students <= 2000)
print(f"8. Is enrollment <= 2000? {enrollment_below_target}") #
Output: True

# 9. Comparing strings
institute = "Edukron"
is_edukron = (institute == "Edukron")
print(f"9. Is institute name 'Edukron'? {is_edukron}") # Output: True

# 10. Comparing booleans themselves
are_flags_equal = (is_data_science_active == is_python_essential) #
True == True
print(f"10. Are DS active and Python essential flags equal?
{are_flags_equal}") # Output: True
```

b) Boolean Operators (`and` , `or` , `not`)

Explanation: Combining boolean values.

- `x and y`: `True` only if both `x` and `y` are `True`. Short-circuits (if `x` is `False`, `y` is not evaluated).
- `x or y`: `True` if at least one of `x` or `y` is `True`. Short-circuits (if `x` is `True`, `y` is not evaluated).
- `not x`: `True` if `x` is `False`, `False` if `x` is `True`. **Examples:**

```
is_data_science_active = True
is_azure_devops_course_full = False
average_rating = 4.8
min_rating = 4.5
has_prerequisites = True

# 1. AND: Check if DS is active AND rating is good
can_recommend_ds = is_data_science_active and (average_rating >
min_rating)
print(f"1. Can recommend Data Science course? {can_recommend_ds}") #
Output: True (True and True)

# 2. AND: Check if ADO is full AND rating is good (Short-circuits)
should_waitlist_ado = is_azure_devops_course_full and
(average_rating > min_rating)
print(f"2. Should waitlist for Azure DevOps? {should_waitlist_ado}")
# Output: False (False and ...)

# 3. OR: Check if DS is active OR ADO is full
show_course_status = is_data_science_active or
is_azure_devops_course_full
print(f"3. Show status (DS active or ADO full)?
{show_course_status}") # Output: True (True or False)

# 4. OR: Check if rating is low OR ADO is full (Short-circuits)
needs_attention = (average_rating < 4.0) or
is_azure_devops_course_full
print(f"4. Does Edukron need attention (low rating or full ADO)?
{needs_attention}") # Output: False (False or False)

# 5. NOT: Check if ADO course is NOT full
can_enroll_ado = not is_azure_devops_course_full
print(f"5. Can enroll in Azure DevOps? {can_enroll_ado}") # Output:
True (not False)

# 6. NOT: Check if DS course is NOT active
is_ds_inactive = not is_data_science_active
print(f"6. Is Data Science course inactive? {is_ds_inactive}") #
Output: False (not True)

# 7. Combining operators: Active course with good rating and
prerequisites met
eligible_to_start = is_data_science_active and (average_rating >=
```

```
min_rating) and has_prerequisites
print(f"7. Eligible to start Data Science? {eligible_to_start}") #
Output: True (True and True and True)

# 8. Combining operators: Not full OR has prerequisites
can_consider = (not is_azure_devops_course_full) or has_prerequisites
print(f"8. Can consider ADO (not full or has prereqs)?
{can_consider}") # Output: True (True or True)

# 9. Complex combination with parentheses
# Active AND (Good rating OR Not Full)
complex_check = is_data_science_active and ((average_rating > 4.7)
or (not is_azure_devops_course_full))
print(f"9. Complex Check Result: {complex_check}") # Output: True
(True and (True or True))

# 10. Using boolean variable directly with 'not'
is_python_essential = True
is_optional = not is_python_essential
print(f"10. Is Python optional? {is_optional}") # Output: False
```

c) `bool()` Constructor and Truth Value Testing

Explanation: The `bool()` function can convert other types to `True` or `False`. In Python, certain values are considered "falsy" (evaluate to `False` in a boolean context), while most others are "truthy" (evaluate to `True`).

- **Falsy values:** `None`, `False`, zero of any numeric type (`0`, `0.0`, `0j`), empty sequences (`''`, `()`, `[]`), empty mappings (`{}`), empty sets (`set()`).
- **Truthy values:** Everything else, including non-empty sequences/mappings, non-zero numbers, `True`. **Examples:**

```
total_students = 1500
average_rating = 4.8
course_list = ["Data Science", "Data Engineering"]
empty_list = []
student_name = "Alice"
no_name = ""
zero_count = 0

# 1. bool() of a non-zero integer
bool_students = bool(total_students)
print(f"1. bool({total_students}): {bool_students}") # Output: True

# 2. bool() of zero
bool_zero = bool(zero_count)
print(f"2. bool({zero_count}): {bool_zero}") # Output: False
```

```python
# 3. bool() of a non-empty list
bool_courses = bool(course_list)
print(f"3. bool(course_list): {bool_courses}") # Output: True

# 4. bool() of an empty list
bool_empty = bool(empty_list)
print(f"4. bool(empty_list): {bool_empty}") # Output: False

# 5. bool() of a non-empty string
bool_name = bool(student_name)
print(f"5. bool('{student_name}'): {bool_name}") # Output: True

# 6. bool() of an empty string
bool_no_name = bool(no_name)
print(f"6. bool(''): {bool_no_name}") # Output: False

# 7. bool() of None
bool_none = bool(None)
print(f"7. bool(None): {bool_none}") # Output: False

# 8. Truth value testing in 'if' (non-empty list is True)
if course_list:
    print("8. Course list is not empty (truthy).")
else:
    print("8. Course list is empty (falsy).")
# Output: Course list is not empty (truthy).

# 9. Truth value testing in 'if' (empty list is False)
if empty_list:
    print("9. This won't print (empty list is falsy).")
else:
    print("9. Empty list evaluated as False.")
# Output: Empty list evaluated as False.

# 10. Truth value testing with 'or' default pattern
# Get student name, or use "Guest" if name is empty (falsy)
current_user = no_name or "Guest"
print(f"10. Current user (empty name): {current_user}") # Output:
Guest
current_user = student_name or "Guest"
print(f"    Current user (non-empty name): {current_user}") # Output:
Alice
```

🧬 Binary Types

Binary types are used to represent data in its raw byte form. This is crucial for handling binary files (like images, executables), network communication, and low-

level data manipulation.

1. `bytes`

Explanation: Represents an **immutable** sequence of bytes (integers between 0 and 255). Created using a `b` prefix before a string literal (e.g., `b'hello'`) or the `bytes()` constructor. String literals within `b''` must typically be ASCII-compatible; for other encodings, use `.encode()`.

Your example:

- `project_name = bytes("Edukron Projects", encoding="utf-8")` (or `b'Edukron Projects'` if pure ASCII)

Common Operations & Methods:

Bytes objects share many sequence operations with strings and lists (indexing, slicing, `len()`, `+`, `*`, `in`) but operate on bytes. Key methods include `decode()` and `hex()`.

a) Creation, Indexing, Slicing

Explanation: Creating bytes objects and accessing individual bytes or sequences of bytes. Individual bytes are returned as integers. **Examples:**

```
# 1. Create bytes using literal (ASCII only)
b_literal = b'Edukron'
print(f"1. Bytes literal: {b_literal}") # Output: b'Edukron'

# 2. Create bytes from string using encode() (recommended for non-
ASCII)
project_name_str = "Edukron Projects©"
project_name_bytes = project_name_str.encode('utf-8')
print(f"2. Bytes from encoded string: {project_name_bytes}") #
Output: b'Edukron Projects\xc2\xa9'

# 3. Create bytes from a list of integers (0-255)
byte_list = [69, 100, 117, 107, 114, 111, 110] # ASCII for 'Edukron'
b_from_list = bytes(byte_list)
print(f"3. Bytes from list of ints: {b_from_list}") # Output:
b'Edukron'

# 4. Access the first byte (returns an integer)
first_byte = b_literal[0]
print(f"4. First byte of {b_literal}: {first_byte} (ASCII:
{chr(first_byte)})") # Output: 69 (ASCII: E)

# 5. Access the last byte
```

```
last_byte = b_literal[-1]
print(f"5. Last byte of {b_literal}: {last_byte} (ASCII:
{chr(last_byte)})") # Output: 110 (ASCII: n)

# 6. Get a slice of bytes (returns bytes object)
slice_bytes = b_literal[0:3] # bytes at index 0, 1, 2
print(f"6. Slice [0:3] of {b_literal}: {slice_bytes}") # Output:
b'Edu'

# 7. Get bytes from index 3 onwards
slice_from_3 = b_literal[3:]
print(f"7. Slice [3:] of {b_literal}: {slice_from_3}") # Output:
b'kron'

# 8. Create empty bytes object
empty_bytes = b''
print(f"8. Empty bytes: {empty_bytes}")

# 9. Create bytes object of a specific size, initialized to null
bytes
null_bytes = bytes(5) # 5 null bytes (\x00)
print(f"9. Null bytes of size 5: {null_bytes}") # Output: b'\x00\x00\
x00\x00\x00'

# 10. Accessing bytes in UTF-8 sequence
# project_name_bytes = b'Edukron Projects\xc2\xa9'
copyright_byte_1 = project_name_bytes[-2]
copyright_byte_2 = project_name_bytes[-1]
print(f"10. UTF-8 bytes for ©: {copyright_byte_1},
{copyright_byte_2}") # Output: 194, 169 (0xC2, 0xA9)
```

b) `decode(encoding='utf-8', errors='strict')`

Explanation: Converts the `bytes` object back into a `str`, using the specified `encoding`. `errors` handles cases where bytes are invalid for the encoding ('strict' raises error, 'ignore' drops them, 'replace' inserts replacement char).

Examples:

```
project_name_bytes = b'Edukron Projects\xc2\xa9'
b_literal = b'Edukron'
invalid_utf8 = b'Edukron\xffInvalid' # \xff is not valid UTF-8 start
byte

# 1. Decode UTF-8 bytes back to string
decoded_project_name = project_name_bytes.decode('utf-8')
print(f"1. Decoded project name (utf-8): '{decoded_project_name}'") #
Output: 'Edukron Projects©'

# 2. Decode simple ASCII bytes (utf-8 is compatible)
decoded_literal = b_literal.decode('utf-8')
```

```python
print(f"2. Decoded ASCII literal (utf-8): '{decoded_literal}'") # Output: 'Edukron'

# 3. Decode simple ASCII bytes using 'ascii' encoding
decoded_ascii = b_literal.decode('ascii')
print(f"3. Decoded ASCII literal (ascii): '{decoded_ascii}'") # Output: 'Edukron'

# 4. Attempt to decode non-ASCII using 'ascii' (UnicodeDecodeError)
try:
    project_name_bytes.decode('ascii')
except UnicodeDecodeError as e:
    print(f"4. Error decoding non-ASCII with 'ascii': {e}")
# Output: Error ... 'ascii' codec can't decode byte 0xc2 in position 16...

# 5. Decode invalid UTF-8 with errors='strict' (UnicodeDecodeError)
try:
    invalid_utf8.decode('utf-8', errors='strict')
except UnicodeDecodeError as e:
    print(f"5. Error decoding invalid UTF-8 (strict): {e}")
# Output: Error ... 'utf-8' codec can't decode byte 0xff in position 7...

# 6. Decode invalid UTF-8 with errors='ignore'
decoded_ignore = invalid_utf8.decode('utf-8', errors='ignore')
print(f"6. Decoded invalid UTF-8 (ignore): '{decoded_ignore}'") # Output: 'EdukronInvalid'

# 7. Decode invalid UTF-8 with errors='replace'
decoded_replace = invalid_utf8.decode('utf-8', errors='replace')
print(f"7. Decoded invalid UTF-8 (replace): '{decoded_replace}'") # Output: 'Edukron﹀Invalid' (﹀ is replacement)

# 8. Decode bytes representing Latin-1 (ISO-8859-1)
# Example: The £ symbol is 0xA3 in Latin-1
pound_latin1 = b'\xA3100'
decoded_pound = pound_latin1.decode('latin-1')
print(f"8. Decoded Latin-1 bytes: '{decoded_pound}'") # Output: '£100'

# 9. Decode empty bytes string
decoded_empty = b''.decode('utf-8')
print(f"9. Decoded empty bytes: '{decoded_empty}'") # Output: ''

# 10. Decode bytes created from list
b_from_list = bytes([69, 100, 117]) # Edu
decoded_list_bytes = b_from_list.decode('ascii')
print(f"10. Decoded bytes from list: '{decoded_list_bytes}'") # Output: 'Edu'
```

c) Other Common Methods (`startswith` , `endswith` , `find` , `replace` , `hex`)

Explanation: Bytes objects have many methods analogous to strings, but operating on bytes. `hex()` provides hexadecimal representation. **Examples:**

```
header = b'EDUKRON_DATA_v1.0'
data = b'\x01\x02\x03\x04Edukron\x05\x06'
footer = b'END_DATA'

# 1. Check if header starts with b'EDUKRON'
starts_edu = header.startswith(b'EDUKRON')
print(f"1. Header starts with b'EDUKRON'? {starts_edu}") # Output:
True

# 2. Check if header ends with b'v1.0'
ends_v1 = header.endswith(b'v1.0')
print(f"2. Header ends with b'v1.0'? {ends_v1}") # Output: True

# 3. Find the position of b'DATA' in header
data_pos = header.find(b'DATA')
print(f"3. Position of b'DATA' in header: {data_pos}") # Output: 8

# 4. Find position of b'Edukron' in data bytes
edu_pos = data.find(b'Edukron')
print(f"4. Position of b'Edukron' in data: {edu_pos}") # Output: 4

# 5. Replace b'v1.0' with b'v1.1' in header (creates new bytes)
new_header = header.replace(b'v1.0', b'v1.1')
print(f"5. New header after replace: {new_header}") # Output:
b'EDUKRON_DATA_v1.1'

# 6. Get hexadecimal representation of data bytes
hex_data = data.hex()
print(f"6. Hex representation of data: {hex_data}")
# Output: 010203044564756b726f6e0506 (E d u k r o n)

# 7. Get hex representation with a separator
hex_data_sep = data.hex(sep=':')
print(f"7. Hex representation with separator: {hex_data_sep}")
# Output: 01:02:03:04:45:64:75:6b:72:6f:6e:05:06

# 8. Concatenate bytes using +
full_message = header + data + footer
print(f"8. Full message (concatenated): {full_message[:20]}...") #
Show first 20 bytes
# Output: b'EDUKRON_DATA_v1.0\x01\x02\x03\x04'...

# 9. Check for membership using 'in'
```

```
is_edu_in_data = b'Edukron' in data
print(f"9. Is b'Edukron' in data? {is_edu_in_data}") # Output: True

# 10. Get length (number of bytes)
data_len = len(data)
print(f"10. Length of data bytes: {data_len}") # Output: 13
```

---

2. `bytearray`

Explanation: Represents a **mutable** sequence of bytes (integers between 0 and 255). It has most of the same methods as `bytes`, but also supports in-place modification of elements and methods like `append`, `extend`, `insert`, `remove` (similar to lists, but operating on bytes). Created using the `bytearray()` constructor.

Your example:

- `course_stats = bytearray([10, 20, 30, 40])`

Common Operations & Methods:

a) Creation and Mutability

Explanation: Creating bytearrays and modifying their contents directly by index or using list-like methods. **Examples:**

```
# 1. Create from a list of integers
course_stats = bytearray([10, 20, 30, 40])
print(f"1. Initial bytearray: {course_stats}") # Output:
bytearray(b'\n\x14\x1e(')

# 2. Create from a bytes object
b_literal = b'Edukron'
ba_from_bytes = bytearray(b_literal)
print(f"2. Bytearray from bytes: {ba_from_bytes}") # Output:
bytearray(b'Edukron')

# 3. Create from a string with encoding
ba_from_str = bytearray("Header", 'utf-8')
print(f"3. Bytearray from string: {ba_from_str}") # Output:
bytearray(b'Header')

# 4. Create an empty bytearray
empty_ba = bytearray()
print(f"4. Empty bytearray: {empty_ba}")

# 5. Create zero-filled bytearray of specific size
zero_ba = bytearray(5)
```

```
print(f"5. Zero-filled bytearray: {zero_ba}") # Output: bytearray(b'\x00\x00\x00\x00\x00')

# 6. Modify a byte at a specific index
print(f"6. Course stats before modify: {course_stats}")
course_stats[0] = 15 # Change first byte from 10 to 15
print(f"   Course stats after modify index 0: {course_stats}")
# Output: bytearray(b'\x0f\x14\x1e(')

# 7. Modify a slice (must assign iterable of bytes/ints)
print(f"7. Course stats before slice assign: {course_stats}")
course_stats[1:3] = [25, 35] # Replace bytes at index 1, 2
# Or use bytes: course_stats[1:3] = b'\x19\x23'
print(f"   Course stats after slice assign: {course_stats}")
# Output: bytearray(b'\x0f\x19#(') (Note: 35 is '#')

# 8. Append a single byte (integer 0-255)
course_stats.append(50)
print(f"8. Course stats after append: {course_stats}")
# Output: bytearray(b'\x0f\x19#(2') (Note: 50 is '2')

# 9. Extend with an iterable of bytes/ints
course_stats.extend([60, 70])
# Or use bytes: course_stats.extend(b'<F')
print(f"9. Course stats after extend: {course_stats}")
# Output: bytearray(b'\x0f\x19#(2<F')

# 10. Attempt to assign integer > 255 (ValueError)
try:
    course_stats[0] = 300
except ValueError as e:
    print(f"10. Error assigning value > 255: {e}")
# Output: Error assigning value > 255: byte must be in range(0, 256)
```

(Bytearray also supports `decode`, `hex`, `startswith`, `endswith`, `find`, `replace`, `pop`, `remove`, `insert`, `len`, `+`, `, in similar to bytes and list`)

---

3. `memoryview`

Explanation: Creates a memory view object that allows accessing the internal data of another object (like `bytes`, `bytearray`) **without making a copy**. This is efficient for large data. If the underlying object is mutable (like `bytearray`), the memoryview can also be used to modify the data in-place.

Your example:

- `memory_snapshot = memoryview(bytes(5))` (Creates a view of 5 null

bytes)

Common Operations & Usage:

Memoryviews support indexing and slicing, which also return memoryviews (or single bytes as integers). They expose the data in various formats using methods like `tobytes()`, `tolist()`, and allow modification if the underlying object permits.

a) Creation, Accessing Data, Slicing

Explanation: Creating memoryviews from binary objects and accessing their data.
Examples:

```
data_bytes = b'Edukron Data'
data_bytearray = bytearray(b'\x01\x02\x03\x04\x05')

# 1. Create memoryview from bytes
mv_bytes = memoryview(data_bytes)
print(f"1. Memoryview from bytes: {mv_bytes}") # Output: <memory at
0x...>

# 2. Create memoryview from bytearray
mv_bytearray = memoryview(data_bytearray)
print(f"2. Memoryview from bytearray: {mv_bytearray}") # Output:
<memory at 0x...>

# 3. Access length via memoryview
print(f"3. Length via mv_bytes: {len(mv_bytes)}") # Output: 12
print(f"   Length via mv_bytearray: {len(mv_bytearray)}") # Output: 5

# 4. Access individual byte (returns integer)
first_byte = mv_bytes[0]
print(f"4. First byte via mv_bytes: {first_byte}") # Output: 69
(ASCII 'E')

# 5. Access slice (returns another memoryview)
mv_slice = mv_bytes[8:12] # Slice for "Data"
print(f"5. Slice via mv_bytes: {mv_slice}") # Output: <memory at
0x...>
# Convert slice to bytes to see content
print(f"   Slice content: {mv_slice.tobytes()}") # Output: b'Data'

# 6. Convert memoryview back to bytes
bytes_copy = mv_bytearray.tobytes()
print(f"6. mv_bytearray to bytes: {bytes_copy}") # Output: b'\x01\
x02\x03\x04\x05'

# 7. Convert memoryview to list of integers
```

```
int_list = mv_bytearray.tolist()
print(f"7. mv_bytearray to list: {int_list}") # Output: [1, 2, 3, 4,
5]

# 8. Access underlying object
underlying_obj = mv_bytes.obj
print(f"8. Underlying object of mv_bytes: {underlying_obj}") #
Output: b'Edukron Data'
print(f"   Is underlying object same as original? {underlying_obj is
data_bytes}") # Output: True

# 9. Slicing the bytearray view
mv_ba_slice = mv_bytearray[1:3] # View of bytes at index 1, 2
print(f"9. Slice via mv_bytearray: {mv_ba_slice.tolist()}") # Output:
[2, 3]

# 10. Creating from the example memory_snapshot
memory_snapshot = memoryview(bytes(5))
print(f"10. Memory snapshot content: {memory_snapshot.tolist()}") #
Output: [0, 0, 0, 0, 0]
```

b) Modifying Data via Memoryview (on Mutable Objects)

Explanation: Using a memoryview to modify the underlying mutable object (like `bytearray`) without copying. **Examples:**

```
mutable_data = bytearray(b'Hello Edukron')
mv = memoryview(mutable_data)

# 1. Modify a single byte via memoryview
print(f"1. Original bytearray: {mutable_data}")
mv[0] = ord('J') # Change 'H' (72) to 'J' (74)
print(f"   Bytearray after mv modification: {mutable_data}")
# Output: bytearray(b'Jello Edukron')

# 2. Modify a slice via memoryview (assign iterable of ints)
mv_slice = mv[6:13] # Slice for "Edukron"
print(f"2. Original slice bytes: {mv_slice.tobytes()}")
mv_slice[:] = b'PYTHON!' # Assign bytes of same length
print(f"   Bytearray after slice modification: {mutable_data}")
# Output: bytearray(b'Jello PYTHON!')

# 3. Modify slice with different length (not allowed for memoryview
slice assignment)
try:
    mv_slice = mv[0:5] # "Jello"
    mv_slice[:] = b'Hi' # Assign shorter sequence
except ValueError as e:
    print(f"3. Error assigning slice of different length: {e}")
# Output: Error ... memoryview assignment: lvalue and rvalue have
```

```
different structures

# NOTE: You *can* modify the underlying bytearray slice directly,
which the view reflects
mutable_data[0:5] = b'Hi' # Modify bytearray directly
print(f"3b. Bytearray after direct slice assign: {mutable_data}") #
Output: bytearray(b'Hi PYTHON!')
print(f"    Memoryview now reflects: {mv.tobytes()}") # Output: b'Hi
PYTHON!' (View is shorter now)

# Reset for next examples
mutable_data = bytearray(b'\x00\x01\x02\x03\x04')
mv = memoryview(mutable_data)

# 4. Modify slice using list of integers
mv[1:4] = [10, 20, 30] # Assign to indices 1, 2, 3
print(f"4. Bytearray after assigning list [10, 20, 30]:
{mutable_data}")
# Output: bytearray(b'\x00\n\x14\x1e\x04')

# 5. Modify using memoryview slice index
mv_s = mv[1:4] # View of bytes 10, 20, 30
mv_s[0] = 11 # Modify first byte of the slice view (index 1 of
original)
print(f"5. Bytearray after modifying slice view index 0:
{mutable_data}")
# Output: bytearray(b'\x00\x0b\x14\x1e\x04')

# 6. Cannot modify memoryview of immutable object (bytes)
immutable_data = b'Immutable'
mv_imm = memoryview(immutable_data)
try:
    mv_imm[0] = ord('X')
except TypeError as e:
    print(f"6. Error modifying memoryview of bytes: {e}")
# Output: Error ... cannot modify read-only memory

# 7. Using memoryview for efficient partial reads/writes (conceptual)
# Imagine reading into a large bytearray buffer
buffer = bytearray(1024)
mv_buffer = memoryview(buffer)
# Read 100 bytes into start of buffer (simulated)
# network_socket.readinto(mv_buffer[0:100])
# Process header from first 10 bytes
# process_header(mv_buffer[0:10].tobytes())
print("7. Conceptual use for efficient I/O shown in comments.")

# 8. Releasing the memoryview (explicitly, though usually handled by
GC)
# Not a direct method, but letting go of references allows GC
```

```python
mv = None
print("8. Memoryview reference set to None.")

# 9. Casting memoryview format (advanced, e.g., view bytes as ints)
# Requires data length to be multiple of item size
int_data = bytearray([0, 0, 0, 1, 0, 0, 0, 2]) # Two 32-bit ints
(little-endian)
mv_int = memoryview(int_data).cast('i') # Cast to signed int format
print(f"9. Memoryview cast to int list: {mv_int.tolist()}") # Output:
[1, 2] (System endianness matters)

# 10. Modifying through cast view
mv_int[0] = 100 # Modify first integer
print(f"10. Underlying bytearray after int modification: {int_data}")
# Output: bytearray(b'd\x00\x00\x00\x02\x00\x00\x00') (100=0x64,
little-endian)
```

That covers all the built-in data types mentioned in your initial code block, with detailed explanations and examples for each.

Okay, let's break down Python's control structures step-by-step, keeping things clear and focused for beginners and intermediates.

✅ 1. Introduction to Control Structures

What are control structures?

Imagine you're following a recipe. You don't always just go straight from the first step to the last. Sometimes you need to check if an ingredient is ready before adding it (a decision), or maybe you need to stir something for 5 minutes (repetition).

In programming, **control structures** are like the recipe's special instructions. They allow you to control the *flow* of your program's execution. Instead of just running code line-by-line from top to bottom, control structures let you:

1. **Make Decisions:** Execute certain blocks of code only if specific conditions are met.
2. **Repeat Actions:** Execute a block of code multiple times.

Why we need decision-making in programs

Programs need to react differently based on different situations, inputs, or internal states. Without decision-making, a program would do the exact same thing every time it runs. Decision-making allows programs to be:

- **Interactive:** Respond differently to user input (e.g., correct vs. incorrect password).
- **Adaptive:** Handle various data scenarios (e.g., process positive numbers differently than negative ones).
- **Robust:** Manage errors or special cases gracefully.
- **Efficient:** Perform actions only when necessary.

Think about simple things:

- Does the user want to save the file? (Yes/No decision)
- Is the entered username correct? (Yes/No decision)
- Is the number even or odd? (Two different paths)
- Is the temperature too high, too low, or just right? (Multiple possible paths)

Decision-making is fundamental to creating useful and intelligent programs.

✅ 2. Conditional Statements

Conditional statements allow your program to execute specific blocks of code based on whether a condition is `True` or `False`.

`if` statement

- **Purpose:** Executes a block of code *only if* a specified condition is `True`.
- **Syntax:**
```
if condition:
    # Code to execute if condition is True
    # This block MUST be indented
# Code here runs regardless (it's outside the if block)
```
- **Example:**
```
age = 20

if age >= 18:
    print("You are eligible to vote.") # This line runs because 20 >= 18 is True

print("Program continues...") # This line runs always
```

`if-else` statement

- **Purpose:** Executes one block of code if a condition is `True` and a *different* block if the condition is `False`. It provides an alternative path.
- **Syntax:**
```
if condition:
    # Code to execute if condition is True
    # Indented block
else:
    # Code to execute if condition is False
    # Indented block
```
- **Example:**
```
temperature = 15

if temperature > 25:
    print("It's a hot day!")
else:
    print("It's not a hot day.") # This line runs because 15 > 25 is False

print("Weather check complete.") # This line runs always
```

`if-elif-else` ladder

- **Purpose:** Used when you need to check multiple conditions in sequence. Python checks each condition (`if`, then `elif`, then `elif`...) until it finds one that is `True`. It executes the corresponding block and then skips the rest of the ladder (including the final `else`). If *none* of the `if` or `elif` conditions are `True`, the `else` block (if present) is executed.

- **Syntax:**

  ```
  if condition1:
      # Code for condition1 True
  elif condition2:
      # Code for condition2 True (only checked if condition1 was False)
  elif condition3:
      # Code for condition3 True (only checked if condition1 and condition2 were False)
  # ... potentially more elif statements
  else:
      # Code to execute if ALL preceding conditions were False
  ```

- **Example:** (Grading System)

  ```
  score = 75

  if score >= 90:
      grade = "A"
  elif score >= 80:
      grade = "B"
  elif score >= 70:
      grade = "C" # This condition is True (75 >= 70)
  elif score >= 60:
      grade = "D"
  else:
      grade = "F"

  print(f"Your score is {score}, which corresponds to grade {grade}.") # Output: Your score is 75, which corresponds to grade C.
  ```

  Key point: Even though `75 >= 60` is also true, the `elif score >= 70:` block was executed first, and the rest of the ladder was skipped.

Nested conditions

- **Purpose:** Placing one conditional statement inside another. This allows for more complex decision-making based on multiple levels of checks.

- **Syntax:** Indentation is crucial to show which `if` / `else` belongs to which level.
```
if outer_condition:
    # Code for outer condition True
    if inner_condition:
        # Code for both outer and inner conditions True
    else:
        # Code for outer True, but inner False
else:
    # Code for outer condition False
```
- **Example:** (User Access Control)
```
is_logged_in = True
is_admin = False

if is_logged_in:
    print("Welcome!")
    if is_admin:
        print("Admin privileges granted.")
    else:
        print("Standard user access.") # This runs: logged in, but not admin
else:
    print("Please log in to access the system.")
```

Real-life examples summary:

- **Grading system:** Using `if-elif-else` to assign grades based on scores.
- **User access control:** Using `if` (and potentially nested `if`) to check login status and user roles (admin, guest, standard user).
- **Traffic light:** `if` color is red, stop; `elif` color is yellow, prepare to stop; `else` (color is green), go.
- **ATM:** `if` sufficient funds, allow withdrawal; `else`, deny.

✅ 3. Loops

What are loops?

Loops are control structures used to repeat a block of code multiple times. Instead of writing the same (or similar) code over and over again, you write it once inside a loop.

Difference between `for` loop and `while` loop

- **`for` loop:** Used when you want to iterate over a *sequence* (like a list,

string, or range of numbers) or any other *iterable* object. You generally know (or the sequence defines) how many times the loop will run. Think of it as "for each item in this collection...".

- **`while` loop:** Used when you want to repeat a block of code *as long as* a certain condition remains `True`. You might not know in advance how many times it will run. Think of it as "while this condition is true...".

- **`for` loop**

  - **Looping through lists, strings, ranges:**

    ```
    # Looping through a list
    fruits = ["apple", "banana", "cherry"]
    print("Fruits:")
    for fruit in fruits:
        print(f"- {fruit}")

    # Looping through a string
    message = "Hello"
    print("\nCharacters in message:")
    for char in message:
        print(char)

    # Looping through a range (explained next)
    print("\nNumbers 0 to 4:")
    for i in range(5): # range(5) generates numbers 0, 1, 2, 3,
    4
        print(i)
    ```

    Output:

    ```
    Fruits:
    - apple
    - banana
    - cherry

    Characters in message:
    H
    e
    l
    l
    o

    Numbers 0 to 4:
    0
    1
    2
    3
    4
    ```

- **Using `range()`:** `range()` is a function that generates a sequence of numbers. It's commonly used with `for` loops.
    - `range(stop)`: Generates numbers from 0 up to (but *not including*) `stop`.
        - `range(5)` -> 0, 1, 2, 3, 4
    - `range(start, stop)`: Generates numbers from `start` up to (but *not including*) `stop`.
        - `range(2, 6)` -> 2, 3, 4, 5
    - `range(start, stop, step)`: Generates numbers from `start` up to (but *not including*) `stop`, incrementing by `step`.
        - `range(1, 10, 2)` -> 1, 3, 5, 7, 9
        - `range(5, 0, -1)` -> 5, 4, 3, 2, 1 (counting down)

```python
print("\nEven numbers under 10:")
for num in range(0, 10, 2):
    print(num) # Output: 0, 2, 4, 6, 8
```

- **Using `enumerate()`:** Sometimes you need both the index and the item while looping through a sequence. `enumerate()` provides this easily.

```python
colors = ["red", "green", "blue"]
print("\nColors with index:")
for index, color in enumerate(colors):
    print(f"Index {index}: {color}")

# You can specify a starting index for enumerate
print("\nColors with index starting from 1:")
for index, color in enumerate(colors, start=1):
     print(f"Item {index}: {color}")
```

Output:

```
Colors with index:
Index 0: red
Index 1: green
Index 2: blue

Colors with index starting from 1:
Item 1: red
Item 2: green
Item 3: blue
```

- **`while` loop**

- **Condition-based repetition:** The loop continues *as long as* the condition is `True`. The condition is checked *before* each iteration.
- **Syntax:**

  ```
  while condition:
      # Code to execute as long as condition is True
      # IMPORTANT: Something inside the loop should eventually
      # make the condition False, otherwise it's an infinite
  loop!
  ```
- **Example:** (Countdown)

  ```
  count = 5
  print("\nCountdown:")
  while count > 0:
      print(count)
      count = count - 1 # Crucial step: modify the variable
  used in the condition

  print("Blast off!")
  ```
  Output:

  ```
  Countdown:
  5
  4
  3
  2
  1
  Blast off!
  ```
- **Infinite loop examples and how to avoid them:** An infinite loop occurs when the `while` loop's condition *never* becomes `False`.
  - **Example (Intentional Infinite Loop - sometimes useful in servers or embedded systems):**

    ```
    # while True:
    #     print("Running forever... (Press Ctrl+C to
    stop)")
    #     # In real applications, there would likely
    be a 'break' condition inside
    ```
  - **Example (Accidental Infinite Loop):**

    ```
    # counter = 0
    # while counter < 5:
    #     print(f"Counter is {counter}")
    #     # MISTAKE: We forgot to increment the
    counter! It will always be 0.
    #     # counter = counter + 1 # <-- This line is
    missing!
    ```

- **How to avoid:**
    1. **Ensure the condition can change:** Make sure that variables used in the `while` condition are modified inside the loop in a way that will eventually make the condition `False`.
    2. **Include a `break` statement:** Use `if` conditions inside the loop to check for an exit scenario and use `break` to get out (covered next).
    3. **Set limits:** If applicable, add a counter to limit the maximum number of iterations.

✅ 4. Loop Control Statements

These statements change the normal execution flow *inside* loops (`for` or `while`).

`break` – to stop the loop

- **Purpose:** Immediately terminates the *current* loop (the innermost loop if nested). Execution resumes at the first statement *after* the loop.
- **Example:** (Find the first number divisible by 7)

```
numbers = [12, 15, 21, 25, 30, 35, 40]
found_number = None # Variable to store the result

print("\nSearching for number divisible by 7:")
for num in numbers:
    print(f"Checking {num}...")
    if num % 7 == 0:
        found_number = num
        print(f"Found! {num} is divisible by 7.")
        break # Exit the loop immediately
    # This print won't happen for 21 after the break
    # print(f"{num} is not the one.")

if found_number:
  print(f"The first number found was: {found_number}")
else:
  print("No number divisible by 7 found in the list.")
```

Output:

```
Searching for number divisible by 7:
Checking 12...
Checking 15...
```

```
Checking 21...
Found! 21 is divisible by 7.
The first number found was: 21
```

`continue` – to skip to next iteration

- **Purpose:** Skips the rest of the code *inside the current iteration* of the loop and proceeds directly to the *next iteration*.
- **Example:** (Print only odd numbers)

```python
print("\nPrinting odd numbers from 0 to 9:")
for i in range(10):
    if i % 2 == 0: # If the number is even...
        continue   # ...skip the rest of this iteration (the print statement)
    # This line is only reached if the number is odd
    print(i)
```

Output:

```
Printing odd numbers from 0 to 9:
1
3
5
7
9
```

`pass` – to write an empty block

- **Purpose:** `pass` is a null operation – nothing happens when it executes. It's used as a placeholder where syntactically some code is required, but you don't want any code to execute. This is common during development when you're outlining structures.
- **Example:**

```python
def my_function():
    # I'll implement this later
    pass # Avoids an IndentationError

for i in range(5):
    if i == 3:
        # Maybe handle 3 specially later?
        pass # For now, do nothing different for 3
    else:
        print(i)

pass # You can put pass anywhere a statement is needed

print("Finished example with pass")
```

✅ 5. Nested Loops

- **Purpose:** Placing one loop inside another loop. The inner loop completes all its iterations for *each single* iteration of the outer loop.
- **Use Case:** Often used for working with 2-dimensional data structures (like grids, matrices, or tables) or when you need to compare items within a list to each other.
- **Example:** (Printing coordinates or iterating a 2D list)

```
print("\nNested loop for coordinates (2x3 grid):")
# Outer loop controls rows (y-coordinate)
for y in range(2): # y will be 0, then 1
    # Inner loop controls columns (x-coordinate)
    for x in range(3): # x will be 0, 1, 2 for EACH y
        print(f"({x}, {y})")
    print("--- End of row ---") # Shows when outer loop
iterates

print("\nNested loop for a 2D list (matrix):")
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

for row in matrix: # Outer loop iterates through rows
([1,2,3], [4,5,6], ...)
    print(f"Processing row: {row}")
    for item in row: # Inner loop iterates through items in
the current row
        print(f"  Item: {item}")
```

Output:

```
Nested loop for coordinates (2x3 grid):
(0, 0)
(1, 0)
(2, 0)
--- End of row ---
(0, 1)
(1, 1)
(2, 1)
--- End of row ---

Nested loop for a 2D list (matrix):
Processing row: [1, 2, 3]
```

```
  Item: 1
  Item: 2
  Item: 3
Processing row: [4, 5, 6]
  Item: 4
  Item: 5
  Item: 6
Processing row: [7, 8, 9]
  Item: 7
  Item: 8
  Item: 9
```

✅ 6. Loop with `else`

This is a less common but sometimes useful feature in Python. The `else` block associated with a loop executes only if the loop completed its iterations *normally*, meaning it wasn't terminated by a `break` statement.

`for...else` behavior

- The `else` block runs after the `for` loop finishes iterating through all items in the sequence.
- If a `break` statement is executed inside the `for` loop, the `else` block is *skipped*.
- **Use Case:** Often used when searching for an item. The `else` block runs if the item was *not* found (because `break` didn't occur).
- **Example:** (Search for an item)

  ```python
  items = ["pen", "pencil", "eraser"]
  item_to_find = "stapler"
  # item_to_find = "pencil" # Try uncommenting this line

  print(f"\nSearching for '{item_to_find}' using for...else:")
  for item in items:
      if item == item_to_find:
          print(f"Found '{item_to_find}'!")
          break # Exit loop since we found it
  else:
      # This block executes only if the loop finished without break
      print(f"'{item_to_find}' was not found in the list.")
  ```

  Output (when `item_to_find = "stapler"`):

```
Searching for 'stapler' using for...else:
'stapler' was not found in the list.
```

Output (when `item_to_find = "pencil"`):

```
Searching for 'pencil' using for...else:
Found 'pencil'!
```

`while...else` behavior

- The `else` block runs after the `while` loop finishes because its condition becomes `False`.
- If the loop is exited using a `break` statement, the `else` block is *skipped*.
- **Example:** (Countdown with `else`)

```python
n = 3
print("\nCountdown with while...else:")
while n > 0:
    print(n)
    n -= 1
    # if n == 1: # Try uncommenting these two lines
    #     break   # This would skip the else block
else:
    # Executes because the while condition (n > 0) became False naturally
    print("Countdown finished normally.")

print("After the loop.")
```

Output (without break):

```
Countdown with while...else:
3
2
1
Countdown finished normally.
After the loop.
```

Output (if `break` at `n == 1`):

```
Countdown with while...else:
3
2
After the loop.
```

✅ 7. List Comprehensions (Intro)

- **Purpose:** A concise and often more readable way to create lists. They essentially combine a `for` loop (and optionally an `if` condition) into a single line. This is considered more "Pythonic" for simple list creation tasks.

- **Syntax:**

  ```
  new_list = [expression for item in iterable if condition]
  # 'if condition' part is optional
  ```

- **Breakdown:**

  1. `expression`: What to do with each item (e.g., `item * 2`, `item.upper()`, `item`). This becomes an element in the new list.
  2. `for item in iterable`: A standard `for` loop header to iterate over the source sequence.
  3. `if condition` (Optional): Filters items from the iterable. Only items for which the condition is `True` are processed by the `expression`.

- **Example:** (Create a list of squares)

  ```
  # Traditional for loop way
  squares_loop = []
  for x in range(10):
      squares_loop.append(x * x)

  # List comprehension way
  squares_comp = [x * x for x in range(10)]

  print("\nList of squares (loop):", squares_loop)
  print("List of squares (comp):", squares_comp)

  # Example with a condition: squares of even numbers only
  even_squares_comp = [x * x for x in range(10) if x % 2 == 0]
  print("Squares of even numbers (comp):", even_squares_comp)
  ```

  Output:

  ```
  List of squares (loop): [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
  List of squares (comp): [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]
  Squares of even numbers (comp): [0, 4, 16, 36, 64]
  ```

- **Note:** This is just an introduction. List comprehensions are powerful but stick to simple ones initially. They are mainly for *creating* lists based on loops.

✅ 8. Practice Problems

Let's apply these concepts with some common beginner exercises.

1. Print even/odd numbers in a range

```
print("\n--- Practice: Even/Odd Numbers ---")
limit = 10
print(f"Numbers from 0 to {limit-1}:")

for number in range(limit):
    if number % 2 == 0:
        print(f"{number} is Even")
    else:
        print(f"{number} is Odd")
```

2. Sum of digits of a number

```
print("\n--- Practice: Sum of Digits ---")
num_to_sum = 12345
original_num = num_to_sum # Keep original for printing
digit_sum = 0

while num_to_sum > 0:
    digit = num_to_sum % 10      # Get the last digit (remainder when
divided by 10)
    digit_sum = digit_sum + digit # Add it to the sum
    num_to_sum = num_to_sum // 10 # Remove the last digit (integer
division by 10)

print(f"The sum of digits in {original_num} is {digit_sum}") #
Output: 15
```

3. Loop through names or marks

```
print("\n--- Practice: Looping through Lists ---")
student_names = ["Alice", "Bob", "Charlie"]
student_marks = [85, 92, 78]

print("Student Names:")
for name in student_names:
    print(f"- {name}")

print("\nStudent Marks:")
for mark in student_marks:
    print(f"- Score: {mark}")

print("\nStudent Names and Marks (using enumerate):")
for index, name in enumerate(student_names):
    mark = student_marks[index] # Assumes lists are parallel and same
```

```
length
    print(f"- {name}: {mark}")

print("\nStudent Names and Marks (using range and index):")
for i in range(len(student_names)): # len() gives the length of the
list
    name = student_names[i]
    mark = student_marks[i]
    print(f"- {name}: {mark}")
```

4. Simple login attempts using `while`

```
print("\n--- Practice: Simple Login ---")
correct_username = "admin"
correct_password = "password123"
max_attempts = 3
attempts = 0
logged_in = False

while attempts < max_attempts:
    username = input("Enter username: ")
    password = input("Enter password: ")
    attempts += 1 # Increment attempt counter

    if username == correct_username and password == correct_password:
        print("Login successful!")
        logged_in = True
        break # Exit the loop on successful login
    else:
        remaining_attempts = max_attempts - attempts
        if remaining_attempts > 0:
            print(f"Incorrect credentials. {remaining_attempts}
attempts remaining.")
        else:
            print("Incorrect credentials. No attempts left.")

# This part runs after the loop finishes (either by break or
condition fail)
if logged_in:
    print("Welcome to the system.")
else:
    print("Access denied. Too many failed attempts.")
```

This covers the core concepts of control structures in Python as outlined. Practice these examples, try modifying them, and build small programs using these building blocks!